

2 Module 2: Introduction

2.1 Components of OS

2.1.1 1. Introduction to OS Components

An operating system (OS) is composed of multiple components that work together to manage hardware resources, provide services to applications, and facilitate user interaction. The primary components include: - **Kernel** (core component) - **Device Drivers** - **Utilities** - **System Libraries** - **User Interface**

Each component plays a distinct role in ensuring the OS functions efficiently.

2.1.2 2. The Kernel

The **kernel** is the **most critical component** of an OS, responsible for core functionalities. It operates at the lowest level, directly interacting with hardware and managing system resources.

2.1.2.1 2.1 Core Functionalities of the Kernel

The kernel performs four major tasks:

2.1.2.1.1 2.1.1 Process Management

- **Definition of a Process:**
 - A **process** is an **active instance** of a program in execution.
 - A **program** is a **passive entity** (e.g., an executable file), while a **process** is dynamic and consumes system resources (CPU, memory, I/O).
 - Multiple instances of the same program running simultaneously are treated as **independent processes**.
- **Key Responsibilities:**
 - **Process Creation & Termination:** The kernel initiates and halts processes.
 - **Process Scheduling:** Allocates CPU time to processes (e.g., using scheduling algorithms like Round Robin, Priority Scheduling).
 - **Resource Allocation:** Manages CPU, memory, storage, and I/O devices for processes.
 - **Inter-Process Communication (IPC):** Facilitates communication between processes (e.g., pipes, message queues).

2.1.2.1.2 2.1.2 Memory Management

- **Requirement for Execution:**
 - A processor **can only execute instructions and data present in main memory (RAM)**.
 - Both **user processes** and the **OS itself** must reside in RAM to function.
- **Key Responsibilities:**
 - **Memory Allocation:** Assigns memory blocks to processes.
 - **Memory Protection:** Ensures one process cannot access another's memory without permission.
 - **Virtual Memory:** Uses disk space to extend apparent RAM capacity (e.g., swapping, paging).
 - **Memory Deallocation:** Frees memory when a process terminates.

2.1.2.1.3 2.1.3 Disk Management

- **Purpose:**
 - Main memory (RAM) is **volatile**-data is lost when power is turned off.
 - **Secondary storage (disks)** provides **persistent storage** for files and programs.
- **Key Responsibilities:**
 - **File System Management:** Organizes data into files and directories.

- **Storage Allocation:** Assigns disk space to files and applications.
- **Disk Scheduling:** Optimizes read/write operations (e.g., using algorithms like SCAN, C-LOOK).
- **Data Integrity:** Ensures data is stored and retrieved correctly.

2.1.2.1.4 2.1.4 Device Management

- **Purpose:**
 - Manages **input/output (I/O) devices** (e.g., keyboards, monitors, printers, network cards).
- **Key Responsibilities:**
 - **Device Communication:** Handles data transfer between processes and devices.
 - **Request Handling:** Processes read/write requests from applications.
 - **Buffering & Caching:** Temporarily stores data to improve I/O performance.
 - **Error Handling:** Detects and resolves device-related issues.

2.1.3 3. Device Drivers

2.1.3.1 3.1 Definition

- A **device driver** is a **specialized software program** that enables the OS to interact with a **specific hardware device**.
- Acts as a **translator** between the OS and hardware, converting generic OS commands into device-specific instructions.

2.1.3.2 3.2 Key Characteristics

- **Device-Specific:** Each hardware device (e.g., printer, GPU, Wi-Fi adapter) requires its own driver.
- **Pre-Installed or Manual Installation:**
 - Some drivers come **pre-installed** with the OS (e.g., basic keyboard/mouse drivers).
 - Others must be **manually installed** (e.g., proprietary GPU drivers).
- **Dynamic Updates:** New devices may require updated drivers for compatibility.

2.1.3.3 3.3 Examples of Devices Requiring Drivers

Device	Example Driver
Monitor	Display driver
Keyboard	HID (Human Interface Device)
Printer	Printer driver
Network Card	Ethernet/Wi-Fi driver
Graphics Card	NVIDIA/AMD GPU driver

2.1.4 4. Utilities

2.1.4.1 4.1 Definition

- **Utilities** are **system tools and applications** that:
 - Enhance **system performance**.
 - Improve **functionality**.
 - Ensure **security**.
 - Assist in **system maintenance**.

2.1.4.2 4.2 Common Types of Utilities

2.1.4.2.1 4.2.1 Antivirus Software

- **Purpose:** Protects the system from **malicious software** (malware, viruses, ransomware).
- **Functionality:**
 - Scans incoming files/programs for threats.
 - **Quarantines** or **deletes** malicious entities.
 - Provides **real-time protection**.
- **Examples:**
 - Windows Defender (built into Windows).
 - Third-party tools like Norton, McAfee, Avast.

2.1.4.2.2 4.2.2 File Management Systems

- **Purpose:** Organizes and manipulates files and directories.
- **Functionality:**
 - **File Operations:** Create, copy, move, delete, rename.
 - **Directory Operations:** Create, navigate, delete folders.
 - **Search Functionality:** Locate files by name, type, or content.
- **Examples:**
 - **Windows Explorer** (Windows).
 - **Finder** (macOS).
 - **Nautilus** (Linux).

2.1.4.2.3 4.2.3 Compression Tools

- **Purpose:** Reduces file size to **save storage space** and **speed up transfers**.
- **Functionality:**
 - Uses algorithms (e.g., ZIP, RAR, GZIP) to compress data.
 - Supports **archiving** (combining multiple files into one).
- **Examples:**
 - **WinRAR, WinZip** (Windows).
 - **TAR, GZIP** (Linux).
 - **7-Zip** (Cross-platform).

2.1.4.2.4 4.2.4 Disk Management Tools

- **Purpose:** Optimizes disk usage and performance.
- **Functionality:**
 - **Disk Partitioning:** Divides a disk into logical sections.
 - **Formatting:** Prepares a disk for use (e.g., NTFS, FAT32, ext4).
 - **Cleanup:** Removes unnecessary files (e.g., temporary files, cache).
 - **Defragmentation:**
 - * **Problem:** Files become **fragmented** (scattered across disk), slowing access.
 - * **Solution:** Reorganizes files to **contiguous blocks**, improving read/write speed.
- **Examples:**
 - **Disk Management** (Windows).
 - **GParted** (Linux).
 - **Disk Utility** (macOS).

2.1.4.2.5 4.2.5 Other Utilities

- **System Monitors:** Track CPU, memory, and disk usage (e.g., Task Manager, **top** in Linux).

- **Backup Tools:** Create system/restore points (e.g., Time Machine, `rsync`).
- **Network Utilities:** Diagnose connectivity (e.g., `ping`, `traceroute`).

2.1.5 5. System Libraries

2.1.5.1 5.1 Definition

- **System libraries** are **collections of pre-compiled functions** that:
 - Provide **access to kernel functionalities** for user applications.
 - **Abstract complex operations** (e.g., I/O, math computations) into simple function calls.

2.1.5.2 5.2 Role in the OS

- **Bridge Between Applications and Kernel:**
 - User applications **cannot directly access hardware** (for security reasons).
 - Libraries act as **intermediaries**, requesting kernel services on behalf of applications.
- **Code Reusability:** Prevents redundant coding by offering standardized functions.

2.1.5.3 5.3 Common System Libraries

Library Type	Purpose	Examples
I/O Libraries	Handle input/output operations (e.g., reading/writing files).	<code>stdio.h</code> (C), <code>fstream</code> (C++)
Math Libraries	Perform mathematical computations (e.g., trigonometry, logarithms).	<code>math.h</code> (C), NumPy (Python)
Graphics Libraries	Render images, videos, and GUI elements.	OpenGL, DirectX, SDL
Database Libraries	Manage database operations (e.g., queries, transactions).	SQLite, MySQL Connector
Security Libraries	Implement encryption, authentication, and access control.	OpenSSL, Libcrypt
Networking Libraries	Enable network communication (e.g., sockets, protocols).	<code>socket.h</code> (C), Boost.Asio (C++)

2.1.6 6. User Interface (UI)

2.1.6.1 6.1 Definition

- The **user interface** is the **medium through which users interact with the OS**.
- Enables users to **issue commands** and **receive feedback** from the system.

2.1.6.2 6.2 Types of User Interfaces

2.1.6.2.1 6.2.1 Graphical User Interface (GUI)

- **Definition:** Uses **visual elements** (windows, icons, menus) for interaction.
- **Advantages:**
 - **Intuitive** and **user-friendly**.
 - **WYSIWYG (What You See Is What You Get)**-direct manipulation of objects.
- **Examples:**
 - **Windows Explorer** (Windows).
 - **macOS Finder** (Apple).

- **GNOME/KDE** (Linux).

2.1.6.2.2 6.2.2 Command-Line Interface (CLI)

- **Definition:** Uses **text-based commands** for interaction.
- **Advantages:**
 - **Faster** for advanced users.
 - **Scriptable** (automatable via scripts).
- **Examples:**
 - **Command Prompt** (Windows).
 - **Terminal** (Linux/macOS).
 - **PowerShell** (Windows).

2.1.6.2.3 6.2.3 Hybrid Interfaces

- Some systems offer **both GUI and CLI** (e.g., Linux with GNOME + Terminal).

2.1.6.3 6.3 Common UI Operations

- **File Operations:** Create, delete, rename files.
- **System Configuration:** Change settings (e.g., date/time, network).
- **Application Management:** Launch, monitor, terminate programs.
- **User Management:** Add/remove users, set permissions.

2.1.7 7. Summary of OS Components

Component	Primary Role	Key Examples
Kernel	Core management (processes, memory, disk, devices).	Linux Kernel, Windows NT Kernel
Device Drivers	Enables OS-hardware communication.	Printer drivers, GPU drivers
Utilities	Enhance performance, security, and maintenance.	Antivirus, Disk Defragmenter, WinRAR
System Libraries	Provide pre-built functions for applications.	math.h, OpenGL, SQLite
User Interface	Facilitates user interaction (GUI/CLI).	Windows GUI, Bash (CLI)

2.1.8 8. Conclusion

The **operating system** is a **complex ensemble** of components that work cohesively to: 1. **Manage hardware resources** (via the kernel). 2. **Support diverse devices** (via drivers). 3. **Optimize performance** (via utilities). 4. **Simplify development** (via libraries). 5. **Enable user interaction** (via UI).

Understanding these components is **fundamental** to grasping how an OS functions and how it can be optimized for different computing needs.

2.2 Computer System Architecture

2.2.1 Introduction to Computer System Components

The lecture introduces the **four fundamental elements** of a computer system and their interactions. These elements collectively form the foundation of a functional computing environment.

2.2.2 1. Core Elements of a Computer System

A computer system comprises **four primary components**:

1. **Hardware**
2. **Operating System (OS)**
3. **Application Programs**
4. **Users**

Each component plays a distinct role in enabling computation, resource management, and task execution.

2.2.3 2. Hardware Components

Hardware refers to the **physical components** of a computer system. The key hardware elements include:

2.2.3.1 2.1 Processors (CPUs - Central Processing Units)

- **Function:** Execute instructions from **application programs** and the **operating system**.
- **Role:** Act as the “brain” of the computer, performing arithmetic, logical, and control operations.
- **Characteristics:**
 - May consist of **single-core** or **multi-core** designs.
 - Responsible for **fetch-decode-execute** cycle of instructions.

2.2.3.2 2.2 Main Memory (RAM - Random Access Memory)

- **Function:** Temporarily stores **programs** and **data** currently in use by the CPU.
- **Key Properties:**
 - **Volatile:** Data is lost when the computer is powered off.
 - **Fast access:** Significantly faster than secondary storage.
 - **Shared resource:** Multiple programs compete for RAM allocation.

2.2.3.3 2.3 Secondary Storage (Persistent Storage)

- **Function:** Stores **data and programs permanently**, even when the system is powered off.
- **Examples:**
 - Hard Disk Drives (HDDs)
 - Solid State Drives (SSDs)
 - Optical discs (CDs, DVDs)
 - Flash drives (USB sticks)
- **Characteristics:**
 - **Non-volatile:** Retains data without power.
 - **Slower access** compared to RAM but **higher capacity**.

2.2.3.4 2.4 Input/Output (I/O) Devices

- **Function:** Facilitate **communication** between the computer and the external world (users or other systems).
- **Categories:**

- **Input Devices:** Provide data/input to the computer.
 - * Keyboard
 - * Mouse
 - * Scanner
 - * Microphone
- **Output Devices:** Display or transmit processed data.
 - * Monitor
 - * Printer
 - * Speakers
 - * Projectors
- **Bidirectional Devices:** Serve both input and output functions.
 - * Touchscreens
 - * Network Interface Cards (NICs)

2.2.4 3. Operating System (OS)

2.2.4.1 3.1 Definition

- A **system software** that runs **continuously** on the computer.
- Acts as an **intermediary** between **hardware** and **application programs**.

2.2.4.2 3.2 Key Responsibilities

1. **Hardware Management:**
 - Allocates hardware resources (CPU, RAM, storage, I/O) to application programs.
 - Ensures **efficient and fair** distribution of resources.
2. **Process Management:**
 - Controls the execution of programs (process scheduling, multitasking).
3. **Memory Management:**
 - Tracks RAM usage and allocates/deallocates memory as needed.
4. **File System Management:**
 - Organizes and maintains data on secondary storage.
5. **Security & Protection:**
 - Enforces access controls to prevent unauthorized usage.
6. **User Interface:**
 - Provides mechanisms (e.g., GUI, CLI) for users to interact with the system.

2.2.4.3 3.3 Importance

- Without an OS, **application programs cannot directly access hardware**.
- Ensures **stability, efficiency, and security** in system operations.

2.2.5 4. Application Programs

2.2.5.1 4.1 Definition

- **Software applications** designed to perform **specific tasks** for users.
- Run **on top of the OS** and rely on it for hardware access.

2.2.5.2 4.2 Common Types of Application Programs

Category	Examples	Primary Function
Communication	Email clients (Outlook, Gmail)	Send/receive emails
Web Browsing	Chrome, Firefox, Edge	Access and navigate the internet
Entertainment	Video games (Steam, Epic Games)	Interactive gaming experiences
Productivity	Word processors (MS Word, LibreOffice)	Create/edit text documents
Data Analysis	Spreadsheets (Excel, Google Sheets)	Tabulate data, perform calculations
Multimedia	Graphic editors (Photoshop, GIMP)	Edit images/videos
Media Playback	VLC, Windows Media Player	Play audio/video files
Database Management	MySQL, Oracle, MongoDB	Store, retrieve, and manage large datasets

2.2.5.3 4.3 Characteristics

- **Task-specific:** Each application serves a distinct purpose.
- **User-driven:** Executed based on user commands or automated schedules.
- **Dependent on OS:** Requires OS services to function (e.g., memory allocation, file access).

2.2.6 5. Users

2.2.6.1 5.1 Definition

- **Entities** that interact with the computer system to **consume its output**.
- **Not limited to humans**; can include:
 - **Human users** (e.g., individuals operating a PC).
 - **Machines/computers** (e.g., servers communicating with clients).
 - **Other systems** (e.g., IoT devices, automated scripts).

2.2.6.2 5.2 Roles of Users

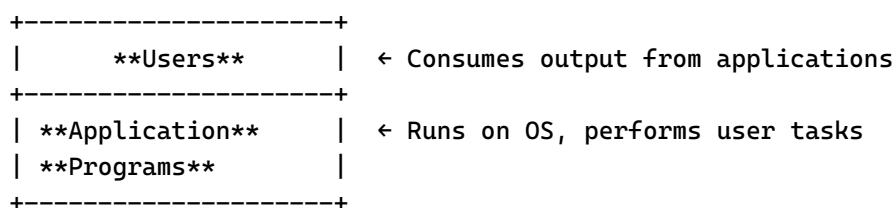
1. **Initiate Tasks:** Launch application programs to perform desired operations.
2. **Provide Input:** Supply data via I/O devices (e.g., typing, file uploads).
3. **Consume Output:** Receive and utilize results generated by applications (e.g., printed documents, displayed videos).

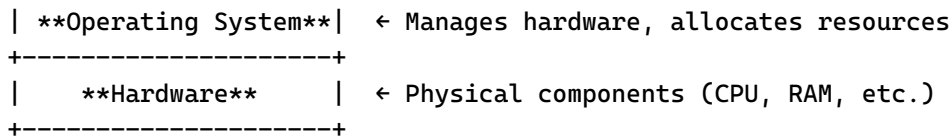
2.2.6.3 5.3 User-Computer Interaction Model

- **Direct Interaction:**
 - Users manually operate applications (e.g., writing an email).
- **Indirect Interaction:**
 - Automated processes (e.g., scheduled backups, cron jobs) act as “users.”

2.2.7 6. Layered Architecture of a Computer System

The interaction between the four components can be visualized in a **hierarchical (layered) model**:





2.2.7.1 6.1 Layer Descriptions

1. **Hardware Layer (Bottom Layer)**
 - Physical components (CPU, RAM, storage, I/O).
 - **Directly controlled by the OS.**
2. **Operating System Layer**
 - **Abstracts hardware complexity** for applications.
 - Provides **APIs/system calls** for applications to request resources.
3. **Application Programs Layer**
 - **Depend on the OS** for execution.
 - **Implement user-specific functionality** (e.g., editing a document).
4. **Users Layer (Top Layer)**
 - **Initiate computations** by running applications.
 - **Receive and utilize** the results.

2.2.7.2 6.2 Interaction Workflow

1. **User** requests a task (e.g., “Open a web browser”).
2. **Application Program** (e.g., Chrome) is launched by the OS.
3. **OS** allocates necessary hardware (CPU time, RAM, network access).
4. **Hardware** executes the application’s instructions.
5. **Output** (e.g., rendered webpage) is returned to the **user**.

2.2.8 7. Summary of Key Concepts

- A computer system consists of **four interdependent layers**: hardware, OS, applications, and users.
- **Hardware** provides the physical infrastructure (CPU, RAM, storage, I/O).
- The **OS** manages hardware resources and enables applications to run.
- **Application programs** perform specific tasks (e.g., gaming, data analysis) on behalf of users.
- **Users** (human or machine) interact with applications to achieve goals.
- The **layered architecture** ensures **modularity, efficiency, and abstraction** in system operations.

2.2.9 8. Practical Implications

- **Resource Allocation**: The OS must **fairly distribute** CPU, RAM, and I/O among applications to prevent crashes or slowdowns.
- **User Experience**: The **design of applications** and the **OS interface** directly impact how users interact with the system.
- **System Stability**: Poor hardware management (e.g., memory leaks) can lead to **system failures**.
- **Security**: The OS must **protect** hardware and data from unauthorized access by users or applications.

2.2.10 9. Real-World Example

Scenario: A user opens a **web browser** (application) to watch a video. 1. **User Action**: Clicks the browser icon. 2. **OS Role**: - Loads the browser into **RAM**. - Allocates **CPU cycles** for execution. - Manages **network I/O** for streaming. 3. **Hardware Involvement**: - **CPU** decodes and executes browser instructions. - **RAM** holds the

browser and video data temporarily. - **Network card** fetches video data from the internet. - **Monitor/speakers** (I/O) display and play the video. 4. **Output**: The user watches the video (consumes the output).

****End of Notes****

2.3 Distributed Systems & Clustered Systems

2.3.1 1. Introduction to Distributed Systems

2.3.1.1 1.1 Definition of a Distributed System

- A **distributed system** is a **collection of independent systems (nodes)** that operate collaboratively.
- **Nodes** can be:
 - Personal computers (PCs)
 - Workstations
 - Mobile phones
 - Tablets
 - Any other standalone computing device
- **Key characteristic**: Each node is **independent and self-sufficient**, capable of functioning on its own.

2.3.1.2 1.2 Communication in Distributed Systems

- Nodes are interconnected via a **communication network**, which can be:
 - **Local Area Network (LAN)** - Used for systems within a single building or campus.
 - **Wide Area Network (WAN)** - Used for systems spread across cities, countries, or globally.
- **No shared main memory**: Unlike multiprocessor systems, distributed systems do **not** have a common memory space.
 - Each node has its **own private memory**.
 - Cooperation is achieved through **message passing** over the network.

2.3.1.3 1.3 Geographical Distribution

- Nodes in a distributed system can be **geographically separated** at various scales:
 - **Same floor in a building**
 - **Different floors in a building**
 - **Different buildings in a campus**
 - **Different campuses in a city**
 - **Across an entire country**
 - **Globally distributed (across continents)**
- The **communication technology** depends on the **degree of separation**:
 - **LAN** for local distributions (e.g., within a building).
 - **WAN** for wide-area distributions (e.g., across cities or countries).

2.3.1.4 1.4 Purpose of Distributed Systems

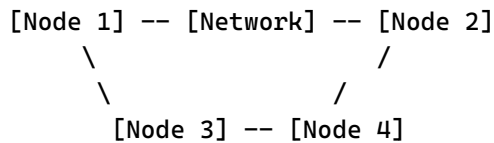
- Nodes **cooperate** to perform tasks **jointly**.
- Goals include:
 - **Resource sharing** (e.g., computational power, storage, data).
 - **Fault tolerance** (continuing operation despite node failures).
 - **Scalability** (adding more nodes to handle increased workload).

- **Transparency** (hiding the complexity of distribution from users).

2.3.1.5 1.5 Architectural Diagram of a Distributed System

- Consists of **multiple standalone nodes** connected via a **communication network**.

- Example:



- Each node operates **independently** but can **communicate and collaborate** with others.

2.3.2 2. Types of Distributed Systems

2.3.2.1 2.1 Client-Server Systems

- **Structure:**
 - **Clients** (e.g., laptops, desktops, tablets) send **requests** to a **server**.
 - **Server** processes requests and returns **responses**.
- **Characteristics:**
 - **Centralized control:** The server is a **single point of service**.
 - **Asymmetric roles:**
 - * Clients **consume** services.
 - * Server **provides** services.
 - **Not fully decentralized** (server is a bottleneck if overloaded).
- **Example:**
 - Web browsing (client = browser, server = web server).
 - Database systems (client = application, server = database).

2.3.2.2 2.2 Peer-to-Peer (P2P) Systems

- **Structure:**
 - **No dedicated server** - all nodes are **equals (peers)**.
 - Any node can act as:
 - * A **service provider (server)**.
 - * A **service consumer (client)**.
- **Characteristics:**
 - **Fully decentralized** - no single point of failure.
 - **Dynamic roles** - a peer can switch between client and server roles.
 - **Scalable** - adding more peers increases system capacity.
- **Examples:**
 - File-sharing networks (e.g., BitTorrent).
 - Blockchain networks (e.g., Bitcoin).
 - Distributed computing (e.g., SETI@home).

2.3.3 3. Introduction to Clustered Systems

2.3.3.1 3.1 Definition of a Clustered System

- A **clustered system** is a type of **multiprocessor system** where:

- **Two or more independent nodes** are combined.
- Nodes are **centrally managed** by an administrator.
- **Key difference from traditional multiprocessor systems:**
 - Traditional multiprocessors share a **single memory space**.
 - Clustered systems **do not share memory** but may share **storage**.

2.3.3.2 3.2 Characteristics of Clustered Systems

- **Centralized management:**
 - A **single administrator** controls all nodes.
 - Unlike distributed systems, which may lack a central authority.
- **High availability & reliability:**
 - **Redundancy** in hardware (multiple processors, storage, network links).
 - **Fault tolerance:** If one node fails, others continue operating (possibly with **performance degradation**).
- **Low geographical separation:**
 - Nodes are typically **within a single campus or data center**.
 - **Not spread across cities or countries** (unlike distributed systems).
- **Low communication latency:**
 - Since nodes are **physically close**, network delays are minimal.
 - Uses **high-speed interconnects** (e.g., InfiniBand, Ethernet).

2.3.3.3 3.3 Comparison with Distributed Systems

Feature	Distributed Systems	Clustered Systems
Geographical Spread	Can be global (WAN) or local (LAN).	Typically within a single campus (LAN).
Management	May lack central control.	Centrally managed by an administrator.
Node Independence	Nodes are standalone and usable independently.	Nodes are not standalone ; part of a cluster.
Communication Latency	Higher (especially in WAN).	Lower (high-speed local networks).
Fault Tolerance	Achieved via redundancy & replication.	Achieved via hardware redundancy.
Use Case	Web services, cloud computing, IoT.	High-performance computing (HPC), databases.

2.3.3.4 3.4 Example: High-Performance Computing (HPC) Clusters

- **Purpose:** Execute **computationally intensive tasks** (e.g., scientific simulations, AI training).
- **How it works:**
 - Multiple nodes **work in parallel** on different parts of a problem.
 - **Load balancing** distributes tasks efficiently.
 - **Shared storage** allows nodes to access common data.
- **Examples:**
 - Supercomputers (e.g., IBM Summit, Fugaku).
 - Research clusters in universities.

2.3.3.5 3.5 Architectural Diagram of a Clustered System

- Consists of **multiple nodes** connected to a **shared storage** system.

- Example:

```
[Computer 1] -- [High-Speed Network] -- [Shared Storage]
[Computer 2] --/
[Computer 3] --/
```
- **Key observations:**
 - Nodes **cannot operate independently** (unlike in distributed systems).
 - **Shared storage** enables coordination.
 - **Central management** ensures synchronization.

2.3.4 4. Summary of Key Concepts

2.3.4.1 4.1 Distributed Systems

- **Definition:** Collection of **independent, standalone nodes** connected via a network.
- **Types:**
 - **Client-Server:** Centralized server handles client requests.
 - **Peer-to-Peer (P2P):** All nodes are equal; no central server.
- **Advantages:**
 - Scalability, fault tolerance, resource sharing.
- **Challenges:**
 - Network latency, security, consistency management.

2.3.4.2 4.2 Clustered Systems

- **Definition:** **Centrally managed** group of nodes working as a **single system**.
- **Key Features:**
 - High availability, low latency, redundancy.
- **Use Cases:**
 - High-performance computing (HPC), databases, web servers.
- **Difference from Distributed Systems:**
 - **Lower geographical spread, centralized control, shared storage.**

2.3.4.3 4.3 When to Use Which?

System Type	Best For	Avoid When
Distributed	Global applications, decentralized control.	Low-latency requirements.
Clustered	High-performance, fault-tolerant computing.	Need for geographically dispersed nodes.

End of Notes

2.4 Dual Mode of Operation

2.4.1 1. Introduction to Dual Mode of Operation

- **Definition:** Dual mode of operation is a fundamental mechanism supported by operating systems (OS) to ensure controlled execution of system services and protection of system resources.
- **Purpose:** It distinguishes between two distinct execution environments:
 - **User Mode:** For executing user applications.

- **Kernel Mode (Supervisor/System/Privileged Mode):** For executing OS services and privileged operations.

2.4.2 2. Need for Dual Mode of Operation

2.4.2.1 2.1 Protection of System Resources

- User applications often require services from the OS, such as:
 - **Input/Output (I/O) operations** (e.g., reading from a file, printing to a screen).
 - **Access to hardware devices** (e.g., disk drives, network interfaces).
- These operations involve **privileged instructions**-instructions that, if executed improperly, could:
 - **Damage other running programs** (e.g., corrupting memory, overwriting data).
 - **Crash the entire system** (e.g., halting the CPU, disabling interrupts).

2.4.2.2 2.2 Preventing Uncontrolled Execution

- **Problem:** If a user program executes privileged instructions without restrictions, it may:
 - **Modify memory of another program**, leading to data loss or corruption.
 - **Disrupt system stability** by altering critical OS structures.
- **Solution:** The OS must enforce a **controlled environment** where privileged operations are restricted to a secure execution mode (kernel mode).

2.4.2.3 2.3 Key Requirements

- **Isolation:** User programs must not interfere with each other or the OS.
- **Controlled Access:** Only the OS (kernel) should execute privileged instructions.
- **Error Handling:** Unauthorized attempts to execute privileged instructions must be detected and terminated.

2.4.3 3. Dual Mode Architecture

2.4.3.1 3.1 Two Modes of Operation

Mode	Alternative Names	Purpose	Mode Bit Value
User Mode	-	Executes user applications (non-privileged code).	1
Kernel Mode	Supervisor Mode, System Mode, Privileged Mode	Executes OS services and privileged instructions .	0

2.4.3.2 3.2 Privileged vs. Non-Privileged Instructions

- **Privileged Instructions:**
 - Can only be executed in **kernel mode**.
 - Examples:
 - * Direct hardware access (e.g., modifying CPU registers, handling interrupts).
 - * Memory management operations (e.g., page table updates).
 - **Consequence of Unauthorized Execution:**
 - * Generates an **interrupt** (trap).
 - * May lead to **abnormal program termination**.
- **Non-Privileged Instructions:**
 - Can be executed in **both user and kernel modes**.

- Examples:
 - * Arithmetic operations (ADD, SUB).
 - * Data movement (LOAD, STORE).

2.4.4 4. Hardware Support for Dual Mode

2.4.4.1 4.1 Mode Bit

- A **hardware bit** (flag) in the CPU that indicates the current execution mode:
 - 1: User mode (unprivileged).
 - 0: Kernel mode (privileged).
- **Purpose:**
 - Enables the CPU to distinguish between user and kernel operations.
 - Prevents user programs from executing privileged instructions.

2.4.4.2 4.2 Transition Between Modes

- **User Mode -> Kernel Mode:**
 - Triggered by:
 - * **System calls** (user program requests an OS service).
 - * **Interrupts** (hardware/software events requiring OS attention).
 - * **Exceptions** (e.g., division by zero, page fault).
 - **Process:**
 1. An **interrupt** is generated.
 2. The **mode bit is set to 0** (switch to kernel mode).
 3. Control transfers to the **kernel space** (memory region for OS code).
- **Kernel Mode -> User Mode:**
 - Occurs after the OS completes the requested service.
 - **Process:**
 1. The **mode bit is set to 1** (switch back to user mode).
 2. Control returns to the **user process**.

2.4.5 5. Implementation of Dual Mode Operation

2.4.5.1 5.1 Execution Flow Example

1. **User Process Execution:**
 - Runs in **user mode** (mode bit = 1).
 - Operates in **user space** (allocated memory for the process).
2. **Service Request:**
 - User program requests a system service (e.g., file I/O).
 - The request involves **privileged instructions**.
3. **Mode Switch:**
 - An **interrupt** is triggered.
 - **Mode bit changes to 0** (kernel mode).
 - Transition to **kernel space**.
4. **Service Location & Execution:**
 - The OS consults the **Interrupt Vector Table (IVT)** to find the **Interrupt Service Routine (ISR)**.
 - The **ISR** executes the requested service (e.g., reading a file).
5. **Return to User Mode:**
 - After service completion, the **mode bit is set to 1**.
 - Control returns to the **user process**.

2.4.5.2 5.2 Key Components

- **Interrupt Vector Table (IVT):**
 - A data structure that maps **interrupt numbers** to **ISR addresses**.
 - Used by the CPU to locate the correct handler for an interrupt.
- **Interrupt Service Routine (ISR):**
 - A kernel function that handles a specific interrupt/service request.
 - Executes in **kernel mode**.
- **User Space vs. Kernel Space:**
 - **User Space:** Memory regions accessible to user processes (limited permissions).
 - **Kernel Space:** Memory regions reserved for the OS (full permissions).

2.4.6 6. Consequences of Violating Dual Mode

- **Attempting Privileged Instructions in User Mode:**
 - The CPU detects the violation via the **mode bit**.
 - **Actions Taken:**
 1. Generates an **interrupt** (illegal instruction trap).
 2. The OS may:
 - * **Terminate the offending process.**
 - * **Log the error** for debugging.
 - * **Notify the user** (e.g., “Segmentation Fault”).
- **Example Scenarios:**
 - A user program tries to **directly access a hardware register**.
 - A program attempts to **modify the memory of another process**.

2.4.7 7. Summary of Dual Mode Benefits

1. **Security:**
 - Prevents malicious or erroneous programs from harming the system.
2. **Stability:**
 - Ensures that only the OS can execute critical operations.
3. **Resource Protection:**
 - Isolates user processes from each other and from the kernel.
4. **Controlled Execution:**
 - System services are provided in a **structured, safe manner**.

2.4.8 8. Real-World Analogies

- **Bank Teller vs. Customer:**
 - **User Mode** = Customer (can request services but cannot access the vault).
 - **Kernel Mode** = Teller (has privileged access to perform transactions).
- **Airport Security:**
 - **User Mode** = Passenger (limited access areas).
 - **Kernel Mode** = Security Staff (full access to restricted zones).

2.4.9 9. Key Terms & Definitions

Term	Definition
Dual Mode	OS feature providing two execution modes: user and kernel.

Term	Definition
Privileged Instruction Mode Bit	CPU instruction restricted to kernel mode (e.g., I/O operations, memory management). Hardware bit indicating current mode (1 = user, 0 = kernel).
User Space	Memory region accessible to user processes.
Kernel Space	Memory region reserved for the OS kernel.
Interrupt	Signal to the CPU indicating an event requiring attention (e.g., system call, hardware event).
Interrupt Vector Table	Data structure mapping interrupts to their handlers (ISRs).
Interrupt Service Routine (ISR)	Kernel function that processes a specific interrupt.

2.4.10 10. Conclusion

- Dual mode of operation is **essential** for modern OS design.
- It ensures **security, stability, and controlled access** to system resources.
- Hardware support (e.g., **mode bit**) and OS mechanisms (e.g., **ISRs, IVT**) work together to enforce this separation.
- Without dual mode, systems would be vulnerable to **crashes, data corruption, and security breaches**.

2.5 Functions of OS System View

2.5.1 Introduction to OS Functions from a System Perspective

- **Learning Objective:** Understand the functionalities of an **Operating System (OS)** from a **system-centric perspective**, focusing on how the OS interacts with hardware and manages resources for applications.

2.5.2 Interaction Between Applications and Hardware via the OS

- Applications in a typical computer environment **do not directly interact with hardware**.
- The **OS acts as an intermediary**, managing and allocating hardware resources to running applications.
- The OS ensures **efficient, fair, and secure** utilization of system resources.

2.5.3 Hardware Components Managed by the OS

The OS interacts with various hardware components, categorized based on their function:

2.5.3.1 1. Input Devices

- **Purpose:** Feed data into the computer system.
- **Examples:**
 - **Keyboard:** Used for typing and entering textual or command-based input.
 - **Mouse:** Provides graphical input (e.g., pointing, clicking, dragging).
- **Communication Direction:**
 - **Unidirectional** (Input -> OS).
 - Data flows **only from the device to the OS**, not vice versa.

2.5.3.2 2. Output Devices

- **Purpose:** Display or produce data generated by the computer.
- **Examples:**

- **Monitor (Display Device):** Outputs visual information (text, graphics, videos) generated by the system.
- **Printer:** Produces hard copies of documents or images for physical use.
- **Communication Direction:**
 - **Unidirectional** (OS → Output).
 - Data flows **only from the OS to the device**, not vice versa.

2.5.3.3 3. Storage Devices

- **Purpose:** Store data **persistently**, even when the system is powered down.
- **Examples:**
 - **Disk Drives (HDD/SSD):** Retain applications, user files, and system data.
- **Communication Direction:**
 - **Bidirectional** (OS ↔ Storage).
 - The OS **reads from and writes to** storage devices.

2.5.3.4 4. Core Processing Components

- **Processor (CPU):** Executes instructions and performs computations.
- **Main Memory (RAM):** Temporarily holds data and instructions for active processes.
- **Communication Direction:**
 - **Bidirectional** (OS ↔ Processor/Memory).
 - The OS **allocates and manages** CPU time and memory space for applications.

2.5.4 OS as a Resource Manager and Allocator

The OS performs **three critical functions** in resource management:

2.5.4.1 1. Resource Management

- The OS **oversees all hardware and software resources** in the system.
- Ensures that resources are **available, accessible, and properly utilized** by applications.

2.5.4.2 2. Resource Utilization Optimization

- **Goal:** Maintain **balanced resource usage** to prevent:
 - **Underutilization:** Resources sitting idle while others are overloaded.
 - **Overutilization:** Some resources being exhausted while others are underused.
- **Impact on Performance:**
 - Optimal utilization ensures **smooth and efficient** system operation.
 - Poor utilization leads to **bottlenecks**, degrading performance.

2.5.4.3 3. Handling Conflicting and Simultaneous Requests

- Multiple applications may **request the same resource simultaneously**.
- The OS must:
 - **Resolve conflicts** (e.g., two processes requesting the same printer).
 - **Prevent inconsistent states** (e.g., data corruption due to improper access).
 - **Ensure fairness** in allocation to avoid **starvation** (an application waiting indefinitely).

2.5.5 OS as a Control Program

- The OS controls the execution of user programs and system applications.
- **Key Responsibilities:**
 - **Prevent Harmful Actions:**
 - * No program (intentionally or unintentionally) should:
 - **Corrupt memory** of another program.
 - **Cause crashes** in other running applications.
 - **Enforce Isolation:**
 - * Programs should operate within **protected memory spaces**.
 - * Unauthorized access to other programs' data/memory is **blocked**.
 - **Ensure Correct Execution:**
 - * Programs run as intended without **interference or disruption** from other processes.

2.5.6 Fairness in Resource Allocation

- The OS must ensure **equitable distribution** of resources among applications.
- **Key Principles:**
 - **No Indefinite Waiting:** Applications should not be **permanently blocked** from accessing a resource.
 - **Priority Handling:** Critical system processes may get **higher priority**, but user applications should still receive **fair access**.
 - **Avoiding Starvation:** Long-running or low-priority processes should eventually **receive necessary resources**.

2.5.7 Summary of OS Functions from a System View

1. **Intermediary Role:** Facilitates communication between **applications and hardware**.
2. **Hardware Interaction:**
 - Manages **input devices** (keyboard, mouse).
 - Controls **output devices** (monitor, printer).
 - Oversees **storage devices** (disks).
 - Allocates **CPU and memory**.
3. **Resource Management:**
 - **Allocates** resources to applications.
 - **Optimizes** utilization to prevent bottlenecks.
 - **Resolves conflicts** in resource requests.
4. **Control Program:**
 - **Protects** programs from harmful interference.
 - **Ensures correct and secure execution**.
5. **Fairness:**
 - **Balances** resource distribution.
 - **Prevents starvation** of applications.

Conclusion: The OS plays a **central role** in managing system resources, ensuring **efficient, fair, and secure** operation of all applications and hardware components.

2.6 Functions of OS User View

2.6.1 Introduction to User-Centric OS Functions

- The lecture examines the **functions of an operating system (OS) from the user's perspective**, focusing on how users interact with and perceive OS operations.

- Users engage with the OS **indirectly through applications** rather than directly with the OS itself.
- This perspective applies to **all types of OS environments**, including:
 - Mainframe OS
 - PC OS
 - Handheld device OS
 - Embedded systems OS

2.6.2 Primary User Concerns in OS Functionality

Users prioritize **three key aspects** when evaluating an OS:

2.6.2.1 1. Convenience

- **Definition:** Ease of use in both **operating the OS itself** and **running applications**.
- **User Focus:**
 - How intuitive is the OS interface?
 - How effortlessly can applications be launched and managed?
 - Example: A user prefers an OS where opening a web browser or a document editor requires minimal steps.

2.6.2.2 2. Usability

- **Definition:** The **practical utility** of the OS in helping users accomplish tasks efficiently.
- **User Focus:**
 - Does the OS provide the necessary tools and features to complete work?
 - Are common tasks (e.g., file management, multitasking) streamlined?
 - Example: A graphic designer values an OS that supports high-performance design software with minimal lag.

2.6.2.3 3. Performance

- **Definition:** The **speed, responsiveness, and reliability** of the OS in executing tasks.
- **User Focus:**
 - Is the system fast enough to meet expectations?
 - Are applications running smoothly without delays?
 - Example: A gamer expects an OS that delivers high frame rates and low latency during gameplay.

2.6.2.4 What Users *Do Not* Prioritize

- **Resource Utilization:**
 - Users generally **do not concern themselves** with how efficiently the OS manages hardware resources (CPU, RAM, disk, etc.).
 - They are **unaware of** (and indifferent to) whether:
 - * Certain hardware components are **overutilized** (e.g., CPU running at 100%).
 - * Other components are **underutilized** (e.g., GPU idle during non-graphical tasks).
 - This indifference holds true **even in single-user systems** (e.g., personal laptops).

2.6.3 Variations in Computing Environments and OS Expectations

OS functionality is **heavily influenced** by the **type of computing environment** it operates in. Different systems have **distinct user expectations** and **design priorities**.

2.6.3.1 1. Shared Usage Systems (Multi-User Environments)

- **Examples:**
 - **Mainframe computers:** Used for **bulk data processing** and **heavy computational tasks** (e.g., banking systems, large-scale simulations).
 - **Mini computers:** Less powerful than mainframes but **more powerful than workstations** (e.g., mid-range servers).
- **Key Characteristics:**
 - **Multi-user access:** Designed for **simultaneous use by multiple users**.
 - **Resource allocation:** OS must **fairly distribute resources** (CPU time, memory, I/O) to prevent:
 - * **Starvation:** Some users being repeatedly deprived of resources.
 - * **Monopolization:** A single user dominating system resources.
 - **User Satisfaction Goal:** The OS must **keep all users content** by ensuring equitable access and performance.

2.6.3.2 2. Dedicated Usage Systems (Single-User Environments)

- **Examples:**
 - **Workstations** (general-purpose desktop computers).
 - **Personal computers (PCs).**
 - **Handheld devices** (smartphones, tablets).
- **Key Characteristics:**
 - **Single-user focus:** OS is optimized for **one primary user**.
 - **No resource-sharing concerns:** The OS does **not need to balance** between multiple users' demands.
 - **Performance Optimization:** Prioritizes **responsiveness and speed** for the sole user's tasks.
 - **Additional Considerations for Handheld Devices:**
 - * **Battery life optimization:** OS must **minimize power consumption** while running applications.
 - * **Memory management:** Efficient use of **limited RAM** to prevent slowdowns.
 - * **User experience (UX):** Enhanced **display, audio, and video** quality for media consumption.
 - Example: A smartphone OS prioritizes smooth scrolling, quick app launches, and long battery life.

2.6.3.3 3. Embedded Systems (Specialized, Minimal-Interaction Environments)

- **Examples:**
 - **Household appliances:** Microwave ovens, dishwashers, OTGs (Oven Toaster Grills).
 - **Automotive systems:** OS controlling engine management, infotainment, or autonomous driving features.
- **Key Characteristics:**
 - **Minimal user interface (UI):** Often **no graphical UI**; may use simple buttons or LED indicators.
 - **Task-specific functionality:** OS is designed to perform **only essential functions** required by the device.
 - * Example: A microwave OS only needs to manage cooking time and power levels-no need for web browsing or multitasking.
 - **No general-purpose computing:** Unlike PCs or smartphones, embedded OSes **do not run arbitrary applications**.

2.6.4 Comparison of OS Functions Across Environments

Environment Type	User Focus	Key OS Priorities	Examples
Shared (Multi-User)	Fairness, resource allocation	Equitable CPU/memory distribution, preventing monopolization, keeping all users happy	Mainframes, mini computers, servers
Dedicated (Single-User)	Convenience, performance, usability	Fast response times, battery optimization (for handhelds), smooth UX	PCs, workstations, smartphones, tablets
Embedded Systems	Reliability, task specificity	Minimal UI, efficient execution of predefined tasks, low power consumption	Microwaves, car systems, IoT devices

2.6.5 Key Takeaways

1. User-Centric OS Functions:

- Users care about **convenience, usability, and performance**-not **resource utilization**.
- The OS must align with **user expectations**, which vary by environment.

2. Environment-Dependent OS Design:

- **Shared systems (mainframes, servers):** Focus on **fair resource allocation** and **multi-user satisfaction**.
- **Dedicated systems (PCs, smartphones):** Optimize for **single-user performance** and **battery/memory efficiency**.
- **Embedded systems:** Prioritize **task-specific reliability** with **minimal UI overhead**.

3. No One-Size-Fits-All OS:

- An OS designed for a **mainframe** cannot (and should not) behave like one for a **smartphone**.
- Each environment imposes **unique constraints and priorities** that shape OS functionality.

4. User Experience vs. System Efficiency:

- While users **do not monitor resource usage**, the OS must **internally manage resources** to meet user expectations (e.g., speed, battery life).

2.6.6 Conclusion

The **functions of an OS from the user's perspective** are dictated by the **computing environment** and the **type of usage** (shared vs. dedicated vs. embedded). Users evaluate an OS based on **ease of use, practicality, and performance**, while the OS must **adapt its behavior** to fulfill these expectations efficiently. Understanding these distinctions is crucial for designing OSes that **meet user needs** across diverse systems.

2.7 Multicore Systems

2.7.1 1. Introduction to Multicore Systems

2.7.1.1 1.1 Definition and Relation to Multiprocessor Systems

- **Multicore systems** are a **variation** (or **logical extension**) of **multiprocessor systems**.
 - In **multiprocessor systems**, multiple **separate processor chips** are present, each containing a **single computing core**.
 - In **multicore systems**, multiple **computing cores** are integrated onto a **single processor chip**.
- **Key Distinction:**
 - **All multicore systems are multiprocessor systems** (since they contain multiple processing units).
 - **Not all multiprocessor systems are multicore** (some may use multiple single-core chips).
- **Throughput Comparison:**

- Multicore systems provide **similar throughput** to traditional multiprocessor systems but with **architectural and performance advantages**.

2.7.2 2. Benefits of Multicore Systems

Multicore systems offer two primary advantages over traditional multiprocessor systems:

2.7.2.1 2.1 Reduced Communication Latency

- **Inter-core communication** (communication between cores) is **faster** when cores are on the **same chip** (on-chip communication) compared to communication across **separate processor chips** (off-chip communication).
 - **Reason:** On-chip communication eliminates delays caused by **interconnects between multiple chips**, reducing **latency**.
 - **Impact:** Faster data exchange between cores improves **parallel processing efficiency**.

2.7.2.2 2.2 Lower Power Consumption

- Multicore systems **consume less power** than equivalent multiprocessor systems (those with the same number of cores but distributed across multiple chips).
 - **Reason:**
 - * **Shared resources** (e.g., memory controllers, bus interfaces) reduce redundant power usage.
 - * **Proximity of cores** minimizes energy loss in signal transmission.
 - **Impact:** Improved **energy efficiency**, which is critical for **mobile devices, data centers, and embedded systems**.

2.7.3 3. Architecture of Multicore Systems

2.7.3.1 3.1 Basic Structural Components The architecture of a multicore system consists of the following key elements:

1. **Processor Chips:**
 - Each **physical processor chip** contains **multiple computing cores** (e.g., dual-core, quad-core).
 - Example: A system may have **two processor chips**, each with **two cores** (total of **four cores**).
2. **Computing Cores:**
 - Each core operates as an **independent processing unit** capable of executing instructions.
 - Cores are identified by unique labels (e.g., **Core 0, Core 1, Core 2, Core 3**).
3. **Private Resources per Core:**
 - **Registers:** Each core has its own set of **registers** for fast data storage and manipulation.
 - **Cache Memory:** Each core includes **dedicated cache memory** (L1, L2, or L3) to reduce access latency to frequently used data.
4. **Shared Main Memory:**
 - All cores (across all processor chips) **share access to a common main memory (RAM)**.
 - **Implications:**
 - Enables **data sharing** between cores.
 - Requires **synchronization mechanisms** (e.g., locks, semaphores) to prevent race conditions.

2.7.3.2 3.2 Example Architecture Diagram A hypothetical multicore system may be structured as follows:

- **Processor 0 (Single Chip):**
 - **Core 0:** Private registers + cache.
 - **Core 1:** Private registers + cache.

- **Processor 1 (Single Chip):**
 - **Core 2:** Private registers + cache.
 - **Core 3:** Private registers + cache.
- **Shared Main Memory:** Accessible by all four cores.

2.7.3.3 3.3 Key Observations

- **Scalability:** Adding more cores to a single chip increases **parallel processing capability** without proportionally increasing power consumption.
- **Memory Bottleneck:** While cores have private caches, **shared main memory** can become a **bottleneck** if not managed efficiently (e.g., via **cache coherence protocols**).
- **Homogeneous vs. Heterogeneous Cores:**
 - **Homogeneous multicore:** All cores are identical (common in general-purpose CPUs).
 - **Heterogeneous multicore:** Cores may differ in design (e.g., some optimized for performance, others for power efficiency), seen in **systems-on-chip (SoCs)**.

2.7.4 4. Summary of Key Concepts

Concept	Description
Multicore System	Multiple computing cores on a single processor chip .
Multiprocessor System	Multiple separate processor chips , each with one or more cores.
On-Chip Communication	Faster than off-chip (between separate processors).
Power Efficiency	Multicore systems consume less power than equivalent multiprocessor systems.
Private Resources	Each core has its own registers and cache .
Shared Memory	All cores access a common main memory , requiring synchronization mechanisms.

2.7.5 5. Conclusion

- Multicore systems **extend the multiprocessor paradigm** by integrating multiple cores onto a single chip, offering **performance and power efficiency benefits**.
- **Architectural advantages** include **reduced communication latency** and **lower energy consumption**.
- **Design challenges** include **memory sharing, cache coherence, and load balancing**, which are critical for optimizing performance.

These notes provide a **comprehensive foundation** for understanding multicore systems, their benefits, and their architectural design. Further exploration may include **parallel programming models, cache coherence protocols, and real-world multicore processors (e.g., Intel Core i7, AMD Ryzen)**.

2.8 Multiprocessor Systems

2.8.1 Introduction to Multiprocessor Systems

- **Definition:** A multiprocessor system is a computer system that contains **multiple CPUs (Central Processing Units) or processors** within a single system.
 - Contrasts with **single-processor systems**, which contain only one CPU.
 - Modern systems (desktops, workstations, mobile phones, tablets) are predominantly multiprocessor systems.
 - Single-processor systems are **obsolete** in contemporary computing.
- **Key Characteristics:**
 - Multiple processors **share access** to:

- * **Main memory (RAM)**
- * **Peripheral devices** (e.g., mouse, monitor, keyboard, disk storage).
- Each processor has its **own set of registers and cache memory** for independent operation.
- **Parallel execution:** If there are n processors, up to n processes can execute **simultaneously**, increasing **throughput** (work done per unit time).

2.8.2 Benefits of Multiprocessor Systems

2.8.2.1 1. Higher Throughput

- **Definition:** Throughput refers to the **amount of work completed per unit of time**.
- **Advantage:**
 - A single processor executes **one process at a time**.
 - n processors can execute n processes **concurrently**, significantly increasing throughput.
 - Example: A system with 10 processors can handle 10 times the workload of a single-processor system in the same timeframe.

2.8.2.2 2. Economic Efficiency

- **Cost Savings:**
 - A multiprocessor system with n processors shares:
 - * **One set of peripherals** (monitor, keyboard, mouse).
 - * **One main memory pool**.
 - * **Optional shared disk storage**.
 - **Comparison with Single-Processor Systems:**
 - * n single-processor systems require:
 - n sets of peripherals.
 - n separate main memory units.
 - * **Result:** Multiprocessor systems are **more cost-effective** due to resource sharing.

2.8.2.3 3. Enhanced Reliability

- **Definition:** Reliability refers to the system's ability to **continue functioning despite failures**.
- **Two Key Aspects:**
 1. **Graceful Degradation:**
 - **Definition:** The system's ability to **maintain partial functionality** when some components fail.
 - **Mechanism:**
 - * If one processor fails, its tasks are **redistributed** among the remaining healthy processors.
 - * Performance **degrades gracefully** (slowdown occurs, but the system remains operational).
 - **Example:**
 - * System with 10 processors, each handling 5 tasks (total: 50 tasks).
 - * If 1 processor fails, its 5 tasks are redistributed:
 - 5 processors now handle 6 tasks each.
 - 4 processors continue handling 5 tasks each.
 - * **Outcome:** System slows down but **does not crash**.
 2. **Fault Tolerance:**
 - **Definition:** The system's ability to **continue normal operation** despite faults (e.g., processor failures).
 - **Mechanism:**
 - * Redundancy of processors ensures that the failure of **one or more processors** does not halt the entire system.

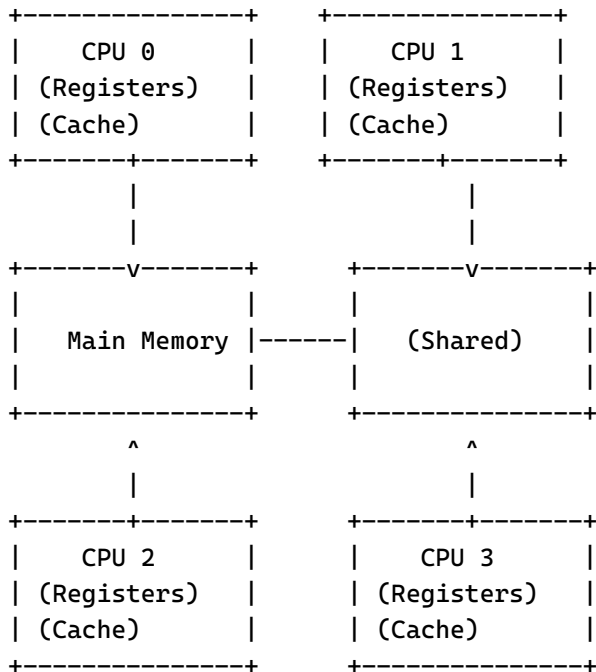
- * Critical tasks can be **reassigned** to functional processors.

2.8.3 Architecture of Multiprocessor Systems

2.8.3.1 1. Shared-Memory Architecture

- **Core Components:**
 - **Multiple Processors (CPUs):** Typically labeled as CPU 0, CPU 1, CPU 2, etc.
 - **Shared Main Memory:** All processors **access the same physical memory**.
 - **Private Components per Processor:**
 - * **Registers:** For storing temporary data during execution.
 - * **Cache Memory:** For reducing latency in memory access.

- **Diagrammatic Representation:**



2.8.3.2 2. Key Design Considerations

- **Load Balancing:**
 - **Definition:** Ensuring **even distribution of workload** across all processors.
 - **Problem:** Imbalanced load (e.g., 8 processors busy, 2 idle) leads to **underutilization**.
 - **Solution:**
 - * Dynamically **assign tasks** to processors to optimize resource usage.
 - * Goal: All processors should have **similar workloads** (not necessarily identical).
- **Multitasking and Multiprogramming:**
 - **Compatibility:** Multiprocessor systems **support** multitasking and multiprogramming, just like single-processor systems.
 - **Difference:**
 - * Single-processor: **Time-sharing** (rapid switching between tasks).
 - * Multiprocessor: **True parallelism** (multiple tasks execute simultaneously).

2.8.4 Summary of Key Points

1. **Multiprocessor Systems** replace single-processor systems in modern computing (desktops, mobiles, etc.).
2. **Shared Resources:**
 - Main memory and peripherals are shared.
 - Each processor has **private registers and cache**.
3. **Benefits:**
 - **Throughput:** n processors $\rightarrow n \times$ work capacity.
 - **Economics:** Lower cost due to shared peripherals/memory.
 - **Reliability:**
 - **Graceful degradation:** Slowdown but no crash on failure.
 - **Fault tolerance:** System remains operational despite processor failures.
4. **Architecture:**
 - Shared-memory model with private caches/registers per CPU.
 - **Load balancing** is critical for optimal performance.
5. **Multitasking/Multiprogramming:** Supported in both single and multiprocessor systems, but multiprocessor enables **true parallelism**.

2.9 Multiprogramming Systems

2.9.1 1. Introduction to Multiprogramming Systems

2.9.1.1 1.1 Definition and Purpose

- **Multiprogramming systems** are designed to improve **resource utilization** in computing environments by allowing **multiple programs (jobs/processes)** to reside in **main memory** simultaneously.
- The primary goal is to **maximize CPU and I/O device usage**, reducing idle time and increasing system efficiency.

2.9.1.2 1.2 Need for Multiprogramming Systems

2.9.1.2.1 1.2.1 Problem with Single-Program Execution

- In a **single-program execution** environment:
 - A **single program** is loaded into **main memory** and executed by the **CPU**.
 - When the program performs an **I/O operation** (e.g., reading from disk, writing to a file), the **CPU remains idle** while waiting for the I/O to complete.
 - Similarly, while the **CPU is executing** the program, **I/O devices remain idle** if no I/O operations are being performed.
 - **Result: Low resource utilization** (CPU and I/O devices are underused).

2.9.1.2.2 1.2.2 Solution: Multiprogramming

- **Multiprogramming** allows **multiple programs (jobs/processes)** to be **loaded into main memory** simultaneously.
- When one program is **waiting for I/O**, another program can **use the CPU**, ensuring that **no resource remains idle** unnecessarily.
- **Key Benefit: Higher throughput and better resource utilization** (CPU and I/O devices are kept busy).

2.9.2 2. Key Concepts in Multiprogramming Systems

2.9.2.1 2.1 Job Pool

- **Definition:** A storage area in secondary memory (e.g., disk) where submitted jobs (programs/processes) are kept before being loaded into **main memory**.
- **Function:**
 - Acts as a **queue** for jobs waiting to be executed.
 - The **job scheduler** selects jobs from this pool to load into **main memory**.

2.9.2.2 2.2 Job Scheduler

- **Definition:** A component of the **operating system (OS)** responsible for:
 - Selecting **which jobs** from the **job pool** should be **loaded into main memory**.
 - Ensuring that the **total size of selected jobs** $\leq$ **available main memory**.
- **Note:** Not all jobs in the pool can be loaded at once due to **memory constraints**.

2.9.2.3 2.3 CPU Scheduler

- **Definition:** A component of the OS that:
 - **Selects which job (process) in main memory** should be **executed next** on the CPU.
 - Uses a **CPU scheduling algorithm** (e.g., First-Come-First-Served, Round Robin, Priority Scheduling).
- **Function:**
 - Allocates the **CPU** to one job at a time.
 - Switches between jobs when one **blocks (e.g., for I/O)** or **completes**.

2.9.2.4 2.4 Job vs. Process Terminology

- In **multiprogramming systems**, the terms **job** and **process** are often used interchangeably.
 - **Job:** Typically refers to a **program submitted for execution** (used in batch systems).
 - **Process:** A more general term referring to an **instance of a program in execution**.

2.9.3 3. Operation of Multiprogramming Systems

2.9.3.1 3.1 Job Submission and Loading

1. **Jobs are submitted** to the **job pool** (secondary storage).
2. The **job scheduler** selects a **subset of jobs** to load into **main memory** (based on memory availability).
3. The **operating system** remains resident in **main memory** to manage job execution.

2.9.3.2 3.2 CPU Execution and I/O Overlap

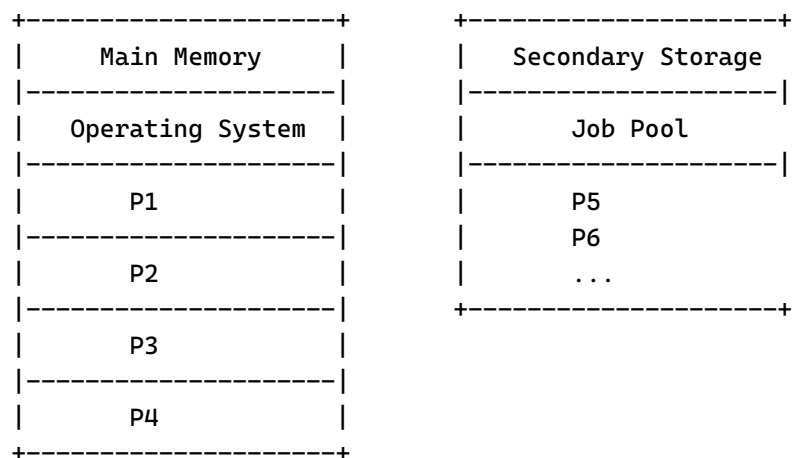
- **Scenario:**
 - **Job P1** is executing on the **CPU**.
 - P1 **requests an I/O operation** (e.g., reading from disk).
 - The **CPU scheduler switches** to another job (**P2**) in memory while P1 waits for I/O.
 - Once P1's I/O completes, it may **resume execution** (if no higher-priority job is running) or wait in a **ready queue**.
- **Key Observations:**
 - **CPU is never idle** (if there are ready jobs in memory).
 - **I/O devices are utilized** while the CPU works on another job.
 - **Resource utilization is maximized** (no wasted CPU or I/O cycles).

2.9.3.3 3.3 Example Workflow

1. **Jobs in Job Pool:** P1, P2, P3, P4, P5, ...

2. **Jobs Loaded into Main Memory:** P1, P2, P3, P4 (selected by job scheduler).
3. **Execution Sequence:**
 - **P1** runs on CPU -> **requests I/O** -> **CPU switches to P2.**
 - **P2** runs on CPU -> **requests I/O** -> **CPU switches to P3.**
 - **P1's I/O completes** -> P1 may **resume** if no higher-priority job is running.
 - **P3** runs -> **completes** -> **CPU switches to P4.**
 - **P2's I/O completes** -> P2 may **resume** later.

2.9.3.4 3.4 Schematic Representation



- **Main Memory:**
 - Contains the OS and a **subset of jobs (P1, P2, P3, P4).**
- **Secondary Storage (Job Pool):**
 - Contains **all submitted jobs (P5, P6, ...)** waiting to be loaded.

2.9.3.5 3.5 Resource Utilization Improvement

Scenario	CPU Usage	I/O Usage	Overall Efficiency
Single Program	Low (idle during I/O)	Low (idle during CPU execution)	Poor
Multiprogramming	High (always executing a job)	High (I/O operations overlap with CPU work)	Excellent

2.9.4 4. Advantages of Multiprogramming Systems

1. **Increased CPU Utilization:**
 - The CPU is **never idle** as long as there is at least one **ready job** in memory.
2. **Better I/O Device Utilization:**
 - I/O operations **do not block the CPU**, as other jobs can run while waiting.
3. **Higher Throughput:**
 - More jobs **complete per unit time** compared to single-program execution.
4. **Efficient Memory Usage:**
 - Multiple jobs **share main memory**, reducing wasted space.
5. **Improved System Responsiveness:**

- Jobs **do not wait unnecessarily** for CPU or I/O, leading to faster turnaround.

2.9.5 5. Potential Challenges

1. Memory Management:

- Must ensure that **total job sizes** $\leq$ **available memory**.
- Requires **memory allocation/deallocation** strategies (e.g., partitioning, paging).

2. CPU Scheduling Overhead:

- Frequent **context switching** between jobs may introduce **overhead**.

3. Job Starvation:

- Some jobs may **never get CPU time** if higher-priority jobs dominate (requires fair scheduling).

4. Deadlocks:

- Poor resource allocation may lead to **circular waits** (e.g., two jobs waiting for each other's resources).

2.9.6 6. Summary of Key Takeaways

- **Multiprogramming** allows **multiple jobs** to reside in **main memory** simultaneously.
- **Job Pool** stores submitted jobs in **secondary memory** before loading into **main memory**.
- **Job Scheduler** selects which jobs to load into memory.
- **CPU Scheduler** decides which job runs next on the CPU.
- **Overlapping CPU and I/O operations** maximizes **resource utilization**.
- **Advantages:** Higher CPU/I/O usage, better throughput, efficient memory usage.
- **Challenges:** Memory management, scheduling overhead, starvation, deadlocks.

End of Notes

2.10 Multitasking Systems

2.10.1 1. Introduction to Multitasking Systems

2.10.1.1 1.1 Overview

- Multitasking systems are a type of operating system design where **multiple processes reside in main memory simultaneously**, similar to multiprogramming systems.
- The key distinction lies in **execution behavior**: in multitasking, the processor **rapidly switches between processes**, executing each for a **very short duration** before moving to the next.
- This creates the **illusion of parallel execution**, even on a **single-processor system**.

2.10.1.2 1.2 Relation to Previous Concepts

- Builds upon **single-processor systems** and **multiprogramming systems** (where multiple programs reside in memory but are not necessarily executed in rapid succession).
- Unlike multiprogramming, multitasking emphasizes **interactivity** and **low response time**.

2.10.2 2. Core Mechanism of Multitasking Systems

2.10.2.1 2.1 Process Execution Cycle

- **Example Scenario:** Four processes (P1, P2, P3, P4) are loaded in main memory.
 - The processor executes **P1 for 3 milliseconds**, then **switches to P2** for another 3 ms.
 - This continues sequentially through **P3 and P4**.
 - If **P1 is not completed** in its initial 3 ms slot, the processor **returns to P1** after completing the cycle (P1 -> P2 -> P3 -> P4 -> P1).

- This **cyclic execution** persists until all processes terminate.

2.10.2.2 2.2 Key Characteristics

- **Short Execution Durations:** Each process gets a **small time slice** (e.g., milliseconds).
- **Rapid Context Switching:** The OS must efficiently **save and restore process states** when switching.
- **Illusion of Concurrency:** Users perceive processes as running **simultaneously**, though only one executes at any given instant.

2.10.2.3 2.3 Overhead and Requirements

- The OS must:
 1. **Select processes** for execution (scheduling).
 2. **Determine execution order** (e.g., round-robin, priority-based).
 3. **Implement a switching mechanism** to transition between processes quickly.
- **Overhead:** The frequent switching introduces **scheduling and context-switching overhead**, but the trade-off enables interactivity.

2.10.3 3. Advantages of Multitasking Systems

2.10.3.1 3.1 Interactivity

- Designed for **interactive systems** where users **directly engage** with running programs.
- Users can:
 - Provide **inputs** to multiple programs.
 - Receive **outputs** in near real-time.
- **Example:** A user can type in a document editor while an antivirus scan runs in the background.

2.10.3.2 3.2 Low Response Time

- **Critical distinction from multiprogramming:**
 - In **multiprogramming**, users have **limited interaction** with executing programs (e.g., batch processing).
 - In **multitasking**, the system **responds quickly** to user inputs (e.g., clicking a browser link while editing an image).
- **User expectation:** Outputs should appear **shortly after input** (e.g., milliseconds to seconds).

2.10.3.3 3.3 Single-Processor Feasibility

- Multitasking **does not require multiple processors**; it can be implemented on a **single CPU**.
- Achieved through **time-sharing** (allocating CPU time in small slices to different processes).

2.10.4 4. Comparison: Multitasking vs. Multiprogramming

Feature	Multitasking Systems	Multiprogramming Systems
Execution Behavior	Rapid switching between processes (short time slices).	Processes reside in memory but may wait longer for execution.
User Interaction	Highly interactive; users engage with multiple programs.	Limited interaction; often batch-oriented.
Response Time	Low (near real-time feedback).	Higher (delays possible due to longer execution queues).

Feature	Multitasking Systems	Multiprogramming Systems
Primary Goal	Create illusion of parallelism for interactivity.	Maximize CPU utilization by keeping it busy.
Example Use Case	Running a web browser, document editor, and antivirus simultaneously.	Batch processing jobs (e.g., payroll calculations).

2.10.5 5. Practical Example: Multitasking in Action

2.10.5.1 5.1 Scenario Description

- **System:** Single-processor machine.
- **Running Processes:**
 1. **Document Editor:** User is typing/editing a file.
 2. **Web Browser:** User is browsing the Internet.
 3. **Antivirus Software:** Scanning the system for malware.
 4. **Image Editor:** User is editing downloaded images.

2.10.5.2 5.2 Execution Flow

- The OS **allocates CPU time** in rapid succession:
 - Executes **document editor** for a few milliseconds -> **switches to web browser** -> **antivirus** -> **image editor** -> cycles back.
- **User Perception:**
 - All four tasks appear to run **simultaneously**.
 - The user can:
 - * Type in the document while the antivirus scans.
 - * Edit an image while the browser loads a webpage.
 - **No noticeable delay** in interaction (assuming efficient scheduling).

2.10.5.3 5.3 Role of the Operating System

- Provides the **environment** for concurrent execution.
- Manages:
 - **Process scheduling** (deciding which process runs next).
 - **Resource allocation** (CPU, memory, I/O).
 - **Context switching** (saving/restoring process states).

2.10.6 6. Key Takeaways

1. **Definition:** Multitasking systems **execute multiple processes in rapid succession**, creating the illusion of parallelism on a single CPU.
2. **Mechanism:**
 - Processes get **short CPU time slices** (e.g., milliseconds).
 - The OS **cycles through processes** repeatedly.
3. **Interactivity:**
 - Users can **interact with multiple programs** concurrently.
 - **Low response time** is critical for user experience.
4. **Distinction from Multiprogramming:**
 - Multiprogramming focuses on **CPU utilization**; multitasking prioritizes **user interaction**.
5. **Implementation:**

- Does **not require multiple CPUs**; works on single-processor systems via **time-sharing**.
6. **Overhead:**
- Requires efficient **scheduling** and **context-switching** mechanisms.

2.10.7 7. Summary of Learning Objectives

By the end of this lecture, students should be able to: - **Define** multitasking systems and their core mechanism. - **Explain** how rapid process switching creates the illusion of concurrency. - **Compare** multitasking and multiprogramming systems. - **Describe** the role of the OS in managing multitasking (scheduling, context switching). - **Provide examples** of multitasking in real-world scenarios (e.g., running multiple applications simultaneously).

2.11 OS Operations Device Controllers & Device Drivers

2.11.1 Introduction to Device Controllers and Device Drivers

- This lecture explores the role of **device controllers** and **device drivers** in enabling communication between hardware devices and the operating system (OS).
- Focuses on how these components facilitate the functioning of peripheral devices (e.g., monitors, keyboards, printers) in a computer system.

2.11.2 Computer System Architecture Overview

2.11.2.1 Components of a Computer System

- Modern computer systems typically consist of:
 - **Processors (CPUs):**
 - * Can be **uni-processor** (single CPU) or **multiprocessor** (multiple CPUs).
 - * Responsible for executing instructions stored in **main memory**.
 - **Device Controllers:**
 - * Hardware components that manage specific types of devices (e.g., disk controllers, USB controllers).
 - * Connect to the **system bus**, which acts as a communication channel between processors, device controllers, and other components.
 - **Main Memory (RAM):**
 - * Shared resource accessed by both **processors** and **device controllers**.
 - * Stores data and instructions required for execution.

2.11.2.2 Concurrent Operation of Processors and Device Controllers

- **Processors** and **device controllers** operate **simultaneously** (not sequentially).
- Both compete for **memory cycles** (access to main memory), leading to potential **resource contention**.
- The OS must manage this competition to ensure efficient system performance.

2.11.3 Device Controllers: Definition and Purpose

2.11.3.1 Why Are Device Controllers Needed?

- Modern computers interact with a **wide variety of peripheral devices**, including:
 - Input devices: **Keyboards, mice, touchscreens**.
 - Output devices: **Monitors, printers**.
 - Storage devices: **Disks, USB drives**.
 - Network devices: **Ethernet adapters, Wi-Fi cards**.
- The **rapid evolution of hardware** makes it impractical for the OS to directly support every possible device.

- **Solution:** Use **device controllers** to standardize communication between the OS and hardware.

2.11.3.2 Definition of a Device Controller

- A **device controller** is a **hardware component** that:
 - Manages **one or more types of devices** (e.g., a **disk controller** manages hard drives).
 - Acts as an **intermediary** between the **hardware device** and the **operating system** (or application programs).
 - Provides a **uniform interface** so the OS does not need to handle device-specific details.

2.11.3.3 Key Functions of a Device Controller

1. **Data Transfer Management:**
 - Receives data from devices (e.g., keyboard input) and stores it in a **local buffer**.
 - Transfers data to/from **main memory** for processing by the CPU.
2. **Device-Specific Operations:**
 - Handles low-level device operations (e.g., reading sectors from a disk, managing USB protocols).
3. **Error Handling:**
 - Detects and reports device errors (e.g., disk read failures, connection issues).

2.11.3.4 Examples of Device Controllers

Device Type	Associated Controller	Function
Disks (HDD/SSD)	Disk Controller	Manages read/write operations to storage.
USB Devices	USB Controller	Handles USB protocols for keyboards, mice, etc.
Monitors	Graphics Adapter (GPU)	Renders and displays visual output.
Network Interfaces	Network Interface Controller (NIC)	Manages data transmission over networks.

2.11.4 Device Drivers: Definition and Purpose

2.11.4.1 Why Are Device Drivers Needed?

- While **device controllers** provide hardware-level management, the OS still needs **software-level support** to interact with devices.
- **Problem:** The OS cannot be updated every time a new device is released.
- **Solution:** Use **device drivers**-modular software components that enable the OS to communicate with specific hardware.

2.11.4.2 Definition of a Device Driver

- A **device driver** is a **software program** that:
 - Acts as a **translator** between the **OS/application** and the **device controller**.
 - Provides an **abstraction layer**, hiding device-specific details from the OS.
 - Allows the OS to **control and utilize** hardware without requiring modifications to the OS kernel.

2.11.4.3 Key Functions of a Device Driver

1. **OS-Hardware Communication:**
 - Converts high-level OS commands (e.g., “print this file”) into low-level device instructions.
2. **Device Initialization:**

- Configures the device during system boot or when the device is connected.
3. **Interrupt Handling:**
 - Manages **interrupts** (signals from devices requesting attention, e.g., a keypress).
 4. **Error Reporting:**
 - Informs the OS of device failures or malfunctions.

2.11.4.4 Example: Installing a New Printer

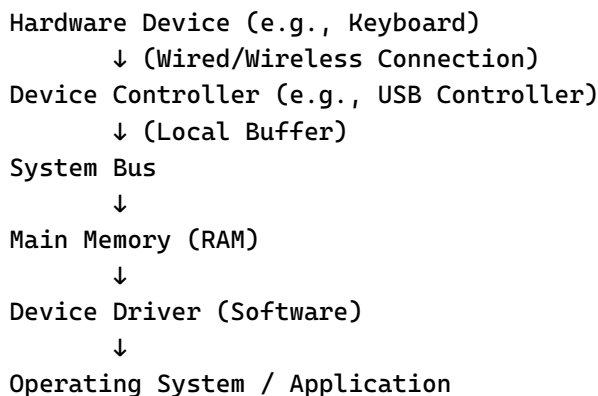
1. A user purchases a **new printer model** not previously supported by the OS.
2. Instead of updating the entire OS, the user installs the **printer's device driver**.
3. The driver enables the OS to:
 - Send print jobs to the printer.
 - Monitor ink levels.
 - Handle paper jams.

2.11.5 Interaction Between Device Controllers and Device Drivers

2.11.5.1 Data Flow from Device to OS

1. **Device -> Device Controller:**
 - An input device (e.g., keyboard) sends data to its **device controller**.
 - The controller stores data in a **local buffer**.
2. **Device Controller -> Main Memory:**
 - The controller transfers data from its buffer to **main memory** via the **system bus**.
3. **OS/Application Access:**
 - The **device driver** interprets the data in memory and makes it available to the OS or application.
 - Example: A keyboard driver converts scan codes into readable characters.

2.11.5.2 Visual Representation of the Process



2.11.5.3 Key Distinctions

Component	Type	Role
Device Controller	Hardware	Manages physical device operations.
Device Driver	Software	Enables OS to communicate with the controller.

2.11.6 System Bus: The Communication Backbone

2.11.6.1 Role of the System Bus

- A **shared communication channel** that connects:
 - CPU(s)
 - Device Controllers
 - Main Memory
- Enables **data transfer** between components.

2.11.6.2 Memory Access Competition

- Both CPUs and **device controllers** require access to **main memory**.
- The OS must **arbitrate access** to prevent conflicts and ensure efficient operation.

2.11.7 Summary of Key Concepts

1. **Device Controllers:**
 - **Hardware** components that manage specific device types.
 - Provide a **standardized interface** for the OS.
 - Examples: Disk controllers, USB controllers, graphics adapters.
2. **Device Drivers:**
 - **Software** components that enable OS-device communication.
 - Allow the OS to support new hardware without modification.
 - Examples: Printer drivers, GPU drivers, network drivers.
3. **Interaction Workflow:**
 - Device -> Controller -> Main Memory -> Driver -> OS/Application.
4. **System Bus:**
 - Facilitates communication between CPU, controllers, and memory.
 - Requires OS management to handle **memory access competition**.
5. **Modularity Benefits:**
 - New devices can be added by installing **drivers** without OS updates.
 - Controllers handle **device-specific details**, reducing OS complexity.

2.11.8 Conclusion

- Device controllers and drivers are **essential** for modern OS operations, enabling seamless integration of diverse hardware.
- **Controllers** provide hardware-level management, while **drivers** offer software-level abstraction.
- This **modular approach** ensures **scalability** and **flexibility** in supporting evolving hardware technologies.

2.12 OS Operations Interrupt Handling

2.12.1 1. Introduction to Interrupts in Operating Systems

- **Context:** This lecture focuses on **interrupt handling**, a critical mechanism in operating systems (OS) that enables efficient interaction between the CPU and I/O devices.
- **Prerequisite Knowledge:** Assumes prior understanding of what an **interrupt** is (briefly revisited here).

2.12.2 2. Concurrent Execution of I/O Devices and Processors

2.12.2.1 2.1 Parallelism in Computer Systems

- **Key Idea:** I/O devices and the CPU execute **concurrently** (simultaneously).
 - Example:
 - * A **process** in **main memory** may be:

- Executed by the **CPU** (running instructions sequentially).
- Performing an **I/O operation** (handled by an I/O device).
- **Implication:** The CPU does not wait idly while I/O operations complete; it switches to other tasks.

2.12.2.2 2.2 Need for Interrupts

- **Scenario:**
 1. A process issues an **I/O request** (e.g., reading from disk).
 2. The **CPU halts execution** of that process and delegates the I/O operation to the corresponding **device controller**.
 3. Upon completion of the I/O operation, the **device controller must notify the CPU** to resume the process.
- **Mechanism:** This notification is achieved via an **interrupt**.
 - **Definition:** An **interrupt** is a signal sent to the CPU by a hardware or software event, indicating that immediate attention is required.

2.12.3 3. Interrupt Handling Workflow

2.12.3.1 3.1 CPU Response to an Interrupt When an interrupt occurs, the CPU: 1. **Pauses** the current execution of the program/instruction. 2. **Saves the state** of the interrupted program (details in Section 3.3). 3. **Identifies the interrupt type** using an **interrupt number**. 4. **Locates the corresponding Interrupt Service Routine (ISR)** via the **Interrupt Vector Table (IVT)**. 5. **Executes the ISR** to handle the interrupt. 6. **Restores the saved state** and resumes the interrupted program.

2.12.3.2 3.2 Interrupt Vector Table (IVT)

- **Definition:** A data structure (table) that maps **interrupt numbers** to the **memory addresses** of their corresponding **Interrupt Service Routines (ISRs)**.
 - **Purpose:** Enables the CPU to quickly locate the correct ISR for a given interrupt.
 - **Contents:**
 - * Each entry in the IVT is a **pointer** to an ISR.
 - * The **interrupt number** acts as an **index** into the IVT.
- **Key Fact:** The number of possible interrupts on a system is **fixed** (hardware-dependent).

2.12.3.3 3.3 Saving and Restoring Program State

- **System Stack:**
 - A dedicated region in **main memory** used to store the **state** of an interrupted program.
 - **Saved Components:**
 1. **CPU Registers:** Temporary storage locations within the CPU (e.g., accumulator, flags).
 2. **Program Counter (PC):** The address of the next instruction to be executed in the interrupted program.
- **Purpose:** Ensures the CPU can **resume execution** of the interrupted program **exactly where it left off** after handling the interrupt.

2.12.3.4 3.4 Execution of the Interrupt Service Routine (ISR)

- **Definition:** A **specialized function** (piece of code) designed to handle a specific type of interrupt.
 - **Examples:**
 - * Handling a **keyboard input**.
 - * Managing a **disk I/O completion**.

- **Workflow:**
 1. The CPU **transfers control** to the ISR (using the address from the IVT).
 2. The ISR **executes** the necessary actions (e.g., reading data from a device).
 3. Upon completion, the ISR **returns control** to the interrupted program.
 4. The CPU **restores the saved state** (from the system stack) and **resumes execution**.

2.12.3.5 3.5 Overhead of Interrupt Handling

- **Definition:** The **time and resources** required to:
 - Pause the current program.
 - Save/restore the program state.
 - Locate and execute the ISR.
- **Importance:**
 - Interrupt handling must be **fast** to minimize delays in system operations.
 - Slow interrupt handling can lead to **perceptible lag** in user interactions or system performance.

2.12.4 4. Step-by-Step Example of Interrupt Handling

2.12.4.1 4.1 Scenario Setup

- A **program** is executing sequentially on the CPU.
- During execution of **Instruction N**, an **interrupt** is triggered (e.g., by an I/O device).

2.12.4.2 4.2 Detailed Steps

1. **Interrupt Received:**
 - The CPU detects the interrupt signal.
2. **Pause Execution:**
 - The CPU stops executing the current instruction.
3. **Save State:**
 - The contents of **CPU registers** and the **Program Counter (PC)** are pushed onto the **system stack**.
4. **Locate ISR:**
 - The CPU uses the **interrupt number** to index into the **IVT** and find the **address of the ISR**.
5. **Execute ISR:**
 - The CPU jumps to the ISR and executes it (e.g., processes the I/O completion).
6. **Restore State:**
 - The CPU pops the saved **registers** and **PC** from the system stack.
7. **Resume Execution:**
 - The CPU returns to the interrupted program and continues from **Instruction N+1**.

2.12.5 5. Summary of Key Concepts

2.12.5.1 5.1 Why Interrupts Are Generated

- To **notify the CPU** of events requiring immediate attention (e.g., I/O completion, hardware errors).
- To enable **efficient multitasking** by allowing the CPU to switch between processes and I/O operations.

2.12.5.2 5.2 How Interrupts Are Handled

1. **Detection:** CPU receives an interrupt signal.
2. **State Preservation:** Saves the current program's state (registers, PC) to the system stack.
3. **ISR Invocation:** Uses the IVT to locate and execute the appropriate ISR.

- 4. **State Restoration:** Restores the saved state and resumes the interrupted program.

2.12.5.3 5.3 Critical Components

Component	Description
Interrupt	Signal to the CPU indicating an event needs attention.
Interrupt Number	Unique identifier for the type of interrupt.
Interrupt Vector Table (IVT)	Table mapping interrupt numbers to ISR addresses.
Interrupt Service Routine (ISR)	Code executed in response to a specific interrupt.
System Stack	Memory region storing the state of interrupted programs.

2.12.5.4 5.4 Performance Considerations

- **Minimizing Overhead:** Fast interrupt handling is crucial to avoid system slowdowns.
- **Hardware Limits:** The number of possible interrupts is fixed by the system architecture.

2.13 OS Services Process management, Memory management, Storage management, IO management, Protection

2.13.1 1. Introduction to Operating System Services

- Operating systems (OS) provide a set of **core services** to manage hardware and software resources efficiently.
- These services are categorized into **four broad areas**:
 1. **Process Management**
 2. **Memory Management**
 3. **Storage Management**
 4. **Protection and Security**

2.13.2 2. Process Management

2.13.2.1 2.1 Definition of a Process

- A **process** is any program or task that is **executed on a CPU**.
- Modern systems often have **multiple processors**, each executing one or more processes.

2.13.2.2 2.2 Key Responsibilities of Process Management The OS must: 1. **Provide an execution environment** for processes. 2. **Allocate resources** (e.g., I/O devices, files, memory) to processes. 3. **Schedule processes** efficiently across available CPUs.

2.13.2.3 2.3 Process Scheduling (CPU Scheduling)

- **Problem:** Multiple processes compete for CPU time.
- **Solution:** The OS decides the **order of execution** to maximize efficiency.
 - Example:
 - * Process **P1** is running but gets **paused** (e.g., due to an I/O request).
 - * Instead of leaving the CPU idle, the OS **switches to Process P2** (context switching).
 - * Once P1's I/O operation completes, it may **resume execution**.

2.13.2.4 2.4 Process Suspension and Resumption

- **Suspension:** Occurs when a process is **interrupted** (e.g., by a hardware interrupt or system call).
 - The OS **saves the process state** (registers, program counter) and **pauses execution**.
- **Resumption:** After handling the interrupt, the OS **restores the process state** and **resumes execution**.

2.13.2.5 2.5 Process Synchronization

- **Problem:** Multiple processes may **access shared resources simultaneously**, leading to **race conditions**.
 - Example: Ten processes reading a file is fine, but if **one process writes**, the others must **wait** to avoid corruption.
- **Solution:** The OS enforces **controlled access** to shared resources (e.g., using **locks, semaphores, or monitors**).

2.13.2.6 2.6 Inter-Process Communication (IPC)

- **Purpose:** Processes often **cooperate** to achieve a common goal (e.g., a web browser spawning multiple threads).
- **Mechanisms:** The OS provides **IPC methods** for processes to exchange data:
 - **Shared memory** (processes read/write to a common memory region).
 - **Message passing** (processes send/receive messages via the OS).
 - **Pipes/sockets** (for communication between local or remote processes).

2.13.3 3. Memory Management

2.13.3.1 3.1 Role of Memory in Execution

- **CPU can only execute instructions stored in** main memory (RAM)**.
- **Hard disks are too slow** for direct CPU access; thus, processes must be **loaded into RAM** before execution.

2.13.3.2 3.2 Key Responsibilities of Memory Management

1. **Allocation:**
 - When a process is **created**, the OS assigns it a **portion of memory**.
2. **Deallocation:**
 - When a process **terminates**, its memory is **reclaimed** for future use.
3. **Tracking Memory Usage:**
 - The OS maintains records of:
 - **Allocated memory** (which process owns which segment).
 - **Free memory** (available blocks for new processes).
4. **Efficient Memory Utilization:**
 - Goal: **Minimize wasted memory** (fragmentation) and **maximize CPU utilization**.

2.13.3.3 3.3 Multiprogramming and Memory Efficiency

- **Problem:** If only **one process** is in memory and it **waits for I/O**, the CPU sits **idle**.
- **Solution:** Keep **multiple processes in memory** so the CPU can **switch** to another process during I/O waits.
 - Example:
 - * Process **P1** requests disk I/O -> CPU switches to **P2** while waiting.
 - * Improves **throughput** and **resource utilization**.

2.13.4 4. Storage Management

2.13.4.1 4.1 File Management

- **Files** are the primary unit of persistent storage (e.g., text, multimedia, spreadsheets).
- **OS Responsibilities:**
 - **Create/delete files and directories (folders).**
 - **Organize files** hierarchically (e.g., /Documents/Reports/).
 - **Manipulate files:**
 - * Copy, move, rename, edit.
 - * Set permissions (read/write/execute).

2.13.4.2 4.2 Mass Storage Management

- **Purpose: Persistent storage** (unlike RAM, which is volatile).
- **Common Storage Devices:** HDDs, SSDs, USB drives.
- **Key Tasks:**
 1. **Free Space Management:**
 - Track **available disk space** for new files.
 - Example: When creating a file, the OS quickly locates free blocks.
 2. **Storage Allocation:**
 - Assign disk blocks to files (e.g., **contiguous, linked, or indexed allocation**).
 3. **Disk Scheduling:**
 - **Problem:** Multiple processes may issue **simultaneous read/write requests**.
 - **Solution:** The OS **orders requests** to optimize disk performance (e.g., **SSTF, SCAN, C-LOOK algorithms**).

2.13.4.3 4.3 I/O Systems Management

- **I/O Devices:** Keyboards, mice, printers, network cards.
- **OS Responsibilities:**
 - **Handle I/O requests** from processes.
 - **Manage device drivers** (software interfaces for hardware).
 - **Coordinate with device controllers** (hardware components that operate I/O devices).

2.13.5 5. Protection and Security

2.13.5.1 5.1 Protection Mechanisms

- **Goal:** Ensure processes **access only authorized resources**.
- **Key Functions:**
 - Prevent **illegal access** (e.g., a process reading another process's memory).
 - Avoid **corruption** from incorrect usage (e.g., writing to a read-only file).
- **Implementation:**
 - **Permissions/Access Control Lists (ACLs):**
 - * Example: A file may be **readable by all** but **writable only by the owner**.
 - **Isolation:** Processes run in **separate memory spaces** to prevent interference.

2.13.5.2 5.2 Security Mechanisms

- **Goal:** Protect the system from **external and internal threats**.
- **Key Functions:**

1. Malware Detection:

- Example: If a **USB drive** contains malicious software, the OS **blocks or alerts** the user.

2. Intrusion Prevention:

- **Firewalls** and **antivirus software** (sometimes bundled with the OS).

3. Authentication & Authorization:

- Verify user identities (e.g., passwords, biometrics).
- Restrict access based on **user roles** (e.g., admin vs. guest).

2.13.5.3 5.3 Differences Between Protection and Security

Protection	Security
Ensures correct resource usage (e.g., no unauthorized writes).	Defends against malicious attacks (e.g., viruses, hackers).
Internal mechanism (e.g., memory segmentation).	External defense (e.g., firewalls, encryption).
Prevents accidental corruption .	Prevents intentional harm .

2.13.6 6. Summary of OS Services

Service	Key Functions
Process Management	Scheduling, synchronization, IPC, suspension/resumption.
Memory Management	Allocation/deallocation, tracking free memory, multiprogramming.
Storage Management	File/directory operations, disk scheduling, I/O handling.
Protection & Security	Access control, malware prevention, authentication.

2.13.6.1 6.1 Why These Services Matter

- **Efficiency:** Maximizes CPU, memory, and I/O utilization.
- **Reliability:** Prevents crashes and data corruption.
- **Security:** Protects against unauthorized access and attacks.
- **Usability:** Provides a **stable environment** for applications and users.

2.14 Recording of Operating Systems Week 1 - Live Session on 26-03-14

2.14.1 1. Course Overview and Structure

2.14.1.1 1.1 Instructor Introduction

- **Instructor:** Satish
 - **Academic Background:**
 - * B.A. in Computer Science Technology
 - * M.Tech from MIT
 - * MBA from Scholar University
 - **Professional Experience:**
 - * 25 years in IT industry
 - * Roles: Developer -> Project Manager -> Senior Project Manager -> Delivery Manager
 - * Currently handles delivery for Fortune 500 clients
 - * 19 years of association with BITS Pilani (teaching on weekends)

2.14.1.2 1.2 Course Plan and Pedagogy

- **Total Contact Sessions:** 10
 - **Sessions 1-5:**
 - * Focus: **Application-oriented discussion** on OS components and usage.
 - * Approach: Select key concepts, discuss real-world applications, and analyze market deployments.
 - **Sessions 6-10:**
 - * Focus: **Solving numerical problems** (critical for the 50-mark final exam).
 - * Method: Instructor solves problems first, followed by collaborative problem-solving with students.

2.14.2 2. Definition and Role of an Operating System (OS)

2.14.2.1 2.1 Core Definition

- **Operating System as a Manager:**
 - Acts as a **single point of contact** for machine coordination.
 - Analogous to **air traffic control (ATC)** at an airport:
 - * Manages interactions between **hardware components** (CPU, memory, I/O devices, kernel).
 - * Ensures **efficient resource allocation** and **task coordination**.

2.14.2.2 2.2 Is an OS Mandatory?

- **Student Responses:**
 - Majority: “Mandatory” (Ganesh, Vedha).
 - Alternative View: “Can work via CLI” (Abhijit, Rahul).
- **Instructor’s Clarification:**
 - **Not strictly mandatory, but highly practical:**
 - * Without an OS:
 - Components must **manually coordinate** (e.g., flights landing/taking off without ATC).
 - **Increased complexity and time consumption.**
 - * **Embedded systems** (e.g., card swipers) may run minimal OS-like firmware.
 - **OS as a Mediator:**
 - * Reduces user/hardware complexity by handling:
 - **Resource management** (CPU, memory, I/O).
 - **Security and access control.**

2.14.3 3. Critical OS Files (Architecture Perspective)

2.14.3.1 3.1 Three Essential Files

- **If deleted, the OS becomes non-functional:**
 1. **Kernel:**
 - **Role:** “Brain” of the OS.
 - **Functions:**
 - * Responds to all system requests.
 - * Decides **who does what and how** (task scheduling, resource allocation).
 - **Location:** Typically in `/Windows/System32/` (Windows) or `/boot/` (Linux).
 2. **User32.dll / User Interface File:**
 - **Role:** Handles **all input activities** (e.g., keyboard, mouse, touch).
 - **Location:** `System32` or `System64` folders.
 3. **GDI (Graphical Device Interface):**
 - **Role:** Manages **output activities** (graphical rendering, display).

- **Note:** Name may vary across OS (e.g., “Display Driver” in some systems).
- **Location:** System directories (path varies by OS).
- **Assignment for Next Class:**
 - Locate these files on personal machines (Windows/Linux/Mac).
 - **Warning:** Never modify or delete these files.

2.14.4 4. OS Categorization by Device Type

2.14.4.1 4.1 Market Distribution

Device Type	Dominant OS	Examples
Handheld Devices	Android	Smartphones, tablets
Personal Computers	Windows	Laptops, desktops
Servers	Unix/Linux	Cloud servers, enterprise systems
Embedded Systems	Custom/RTOS (Real-Time)	ATMs, digital cameras, IoT

2.14.4.2 4.2 Key Observations

- **Android:** Dominates mobile devices (user-friendly, touch-optimized).
- **Windows:** Preferred for PCs (GUI, compatibility with legacy software).
- **Unix/Linux:**
 - Servers: Stability, security, and open-source flexibility.
 - **Historical Context:** Evolved from **UNICS** (1970s) -> **Unix** -> **Linux**.
- **Embedded OS:** Lightweight, task-specific (e.g., ATM machines, GPS).

2.14.5 5. Design Priorities for Modern OS

2.14.5.1 5.1 Student Inputs on OS Design

- **Priorities Suggested by Students:**
 - User experience (Sahil).
 - Process scheduling, kernel management, hardware efficiency (Kumar Saptam).
 - Memory management, performance (General consensus).

2.14.5.2 5.2 Instructor’s Prioritization Framework

1. **Performance:**
 - **Goal:** Maximize speed and responsiveness for all devices.
 - **Example:** Fast boot times, minimal latency in task switching.
2. **Memory Management:**
 - **Challenge:** Limited memory in embedded/mobile devices.
 - **Solution:** Efficient allocation (e.g., caching, virtual memory).
3. **Security:**
 - **Context-Dependent:** Critical for servers/financial systems, less for simple embedded devices.
 - **Trade-off:** Security features may impact performance.

2.14.5.3 5.3 Human vs. OS “Thinking”

- **Misconception:** “OS thinks differently from humans.”
- **Reality:**
 - OS is **designed by humans** to mimic logical decision-making.

- **Example:** OS restricts file downloads to **secure formats** to prevent malware.
- * **Why?** To avoid user frustration (e.g., downloading unopenable files).

2.14.6 6. OS as a User Supporter (Not Adversary)

2.14.6.1 6.1 Does OS Support or Restrict Users?

- **Student Responses:**
 - “Supports users” (Abhijit, majority).
 - “Follows strict rules” (contrarian view).
- **Instructor’s Explanation:**
 - **OS supports users within security boundaries:**
 - * **Example 1:** Blocks unsafe file downloads (e.g., .exe from untrusted sources).
 - * **Example 2:** Caches frequently used apps/files for quicker access.
 - * **Example 3:** Auto-saves documents (e.g., Microsoft Word recovery).

2.14.6.2 6.2 Security as a Boundary

- **Analogy:** OS acts like a **society security guard**:
 - Allows entry only with **validation** (e.g., gate passes, permissions).
 - **Purpose:** Protect the system (and user) from threats.

2.14.7 7. OS and Hardware Interaction

2.14.7.1 7.1 System View of OS

- **OS as a Resource Allocator:**
 - Manages **CPU time, memory space, I/O devices**.
 - **Goal:** Optimize hardware usage for applications.
- **User vs. System Perspective:**
 - **User View:** Focuses on ease of use (e.g., GUI, multitasking).
 - **System View:** Prioritizes **hardware efficiency** and **stability**.

2.14.7.2 7.2 Can Users Force OS Behavior?

- **Example:** C Programming (Register Variables)
 - Code Snippet:


```
int x;                // Stored in RAM
register int y = 20; // *Request* to store in CPU register
```
 - **Reality:**
 - * **register** keyword **suggests** storing y in a CPU register (faster access).
 - * **OS has final authority:** May ignore the request if registers are full or reserved.
 - **Lesson:** Users **cannot force** OS behavior; OS prioritizes **system stability**.

2.14.8 8. Case Study: OS for Paralyzed Users

2.14.8.1 8.1 Hypothetical Scenario

- **Design Challenge:** Create an OS for a **paralyzed stroke patient** who cannot speak.
 - **Requirements:**
 - * Thought-to-text/speech conversion.
 - * Minimal physical interaction (e.g., eye-tracking, brainwave sensors).

- **Real-World Example:** Stephen Hawking’s assistive device (eye-tracking + text-to-speech).

2.14.8.2 8.2 Video Demonstration (Neural Interface)

- **Technology: Electrocorticography (ECoG) grid** implanted on the brain.
 - **Function:** Decodes neural signals from **attempted speech muscles** into text/speech.
 - **Current Limitations:**
 - * **Cost:** Prohibitively expensive for widespread use.
 - * **Reliability:** Risk of malfunction; requires surgical implantation.
- **Student Task:**
 - Research and propose a **low-cost, high-performance OS** for such applications.
 - **Constraints:** Minimal memory usage, robust security, affordability.

2.14.9 9. OS in Embedded and Specialized Systems

2.14.9.1 9.1 Examples of OS-Free Devices

- **Claim:** “No device lacks an OS.”
- **Reality:**
 - **Minimalist Firmware:** Some devices use **bare-metal programming** (no full OS).
 - **Examples:**
 - * Early calculators.
 - * Some IoT sensors (direct hardware control).
 - **Assignment:** Find and document a device that operates without a traditional OS.

2.14.9.2 9.2 Market Gaps and Opportunities

- **Why Aren’t Neural Interfaces Mainstream?**
 - **Barriers:**
 - * **Cost:** High R&D and implantation expenses.
 - * **Ethics:** Invasive procedures raise concerns.
 - * **Use Cases:** Limited to severe disabilities (low mass-market demand).
 - **OS Design Challenge:**
 - * Create a **lightweight OS** for neural interfaces that is:
 - **Affordable** (e.g., open-source, modular).
 - **Secure** (protects brainwave data).
 - **Performant** (real-time response).

2.14.10 10. Historical Context and OS Evolution

2.14.10.1 10.1 Timeline of OS Development

- **1970s:** UNICS (Uniplexed Information and Computing System) -> **Unix**.
- **1980s-1990s:** Rise of **Windows** (PC dominance) and **Linux** (server/open-source).
- **2000s-Present:**
 - **Android** (mobile revolution).
 - **MacOS/iOS** (Apple’s ecosystem).
 - **Embedded Linux** (IoT, ATMs, routers).

2.14.10.2 10.2 Lessons from Failed OS (e.g., Windows Vista)

- **Why Vista Failed:**

- **Resource Allocation Issues:** Poor memory/CPU management.
- **Compatibility Problems:** Driver conflicts, bloated features.
- **Key Takeaway:**
 - OS success depends on **balancing performance, security, and user needs**.

2.14.11 11. Practical Assignments and Next Steps

2.14.11.1 11.1 Immediate Tasks

1. **Locate Critical OS Files:**
 - Find paths to `kernel`, `user32.dll`, and `GDI` on personal machines.
2. **Research OS-Free Devices:**
 - Identify and document a device that operates without a traditional OS.
3. **Neural Interface OS Design:**
 - Propose a **low-cost, secure, performant OS** for paralyzed users.

2.14.11.2 11.2 Preparation for Next Session

- **Focus Areas:**
 - **Application-Oriented Discussions:** How OS concepts apply to real-world systems.
 - **Numerical Problems:** Practice resource allocation, scheduling algorithms.
- **Resources:**
 - Review lecture recordings for **user/system views** of OS.
 - Explore **Unix/Linux commands** for hands-on understanding.

2.14.12 12. Closing Remarks

- **Philosophy:** “Think **application-first**, not exam-first.”
 - **Exam Success:** Will follow naturally from understanding concepts.
 - **Career Impact:** Ability to **design OS for novel use cases** (e.g., medical, IoT) is invaluable.
- **Final Question:** “*How would you design an OS for a device that doesn’t exist yet?*”
 - **Hint:** Start with **user needs**, then address **hardware constraints**.

2.15 Single Processor Systems

2.15.1 1. Introduction to Single Processor Systems

2.15.1.1 1.1 Definition

- A **single processor system** is a computing system that contains **only one Central Processing Unit (CPU)**.
- Despite having a single CPU, multiple **processes** can reside in the **main memory (RAM)** simultaneously.
- However, **only one process can execute at any given time** due to the presence of a single CPU.

2.15.1.2 1.2 Key Characteristics

- **Single CPU Constraint:** The system’s processing capability is limited to one CPU, restricting parallel execution.
- **Multiple Processes in Memory:** While only one process runs at a time, several processes may be **loaded in memory**, waiting for execution.
- **Sequential Execution:** Processes are executed **one after another**, not concurrently.

2.15.2 2. Operational Behavior of Single Processor Systems

2.15.2.1 2.1 Process Execution

- The **operating system (OS)** manages the execution of processes by allocating the CPU to one process at a time.
- If a process issues an **I/O (Input/Output) request** (e.g., reading from disk, waiting for user input), the CPU may **switch to another process** already loaded in memory.
 - Example:
 - * **Process A** is executing but requests disk I/O -> CPU halts **Process A** and switches to **Process B**.
 - * Once the I/O operation for **Process A** completes, the OS may resume its execution.

2.15.2.2 2.2 System Components Interaction The single CPU interacts with the following system components:

1. **Main Memory (RAM)** - Stores **instructions** and **data** for executing processes. - The CPU fetches instructions from memory to execute them.
2. **I/O Devices** - The CPU communicates with peripherals (e.g., keyboard, disk, network interfaces). - I/O operations may cause the CPU to **wait**, allowing other processes to run.
3. **Operating System (OS)** - Manages **process scheduling**, deciding which process gets CPU time. - Handles **interrupts** (e.g., I/O completion signals) to switch between processes.
4. **Applications/Programs** - Multiple applications may be **loaded in memory**, but only one executes at a time. - Users **cannot interact with multiple programs simultaneously** (no true multitasking).

2.15.3 3. Limitations of Single Processor Systems

2.15.3.1 3.1 Low System Throughput

- **Throughput** (amount of work done per unit time) is **limited** because only one process executes at a time.
- No **parallel processing** -> slower overall performance compared to multi-processor systems.

2.15.3.2 3.2 Lack of Concurrent User Interaction

- Users **cannot run multiple programs simultaneously** in a meaningful way.
 - Example:
 - * A user cannot type in a word processor while a compilation process runs in the background.
 - * The system appears **unresponsive** if a process monopolizes the CPU.

2.15.3.3 3.3 Poor Reliability and Fault Tolerance

- **Single Point of Failure:** If the **only CPU fails**, the **entire system becomes unusable**.
- **No redundancy** -> system **crashes** until the CPU is repaired or replaced.
- **No failover mechanism** (unlike multi-processor systems where other CPUs can take over).

2.15.4 4. Summary of Single Processor System Workflow

1. **Process Loading:** Multiple processes are loaded into **main memory**.
2. **CPU Allocation:** The **OS scheduler** selects one process for execution.
3. **Execution:** The CPU runs the selected process.
4. **I/O or Blocking Event:**
 - If the process requests I/O, the CPU **switches to another process** (context switch).
 - The original process remains **blocked** until the I/O completes.
5. **Completion or Preemption:**
 - The process either **finishes execution** or is **preempted** (e.g., time slice expires).
 - The CPU moves to the next process in the queue.

2.15.5 5. Key Takeaways

- **Single CPU Constraint:** Only one process executes at a time, even if multiple processes are in memory.
- **OS Role:** The operating system manages **process scheduling** and **I/O handling** to optimize CPU usage.
- **Performance Bottleneck:** Low throughput, no true multitasking, and high vulnerability to CPU failure.
- **Use Cases:** Suitable for **simple, non-critical systems** where cost efficiency is prioritized over performance.

End of Notes

2.16 What is an Operating System

2.16.1 Definition of an Operating System (OS)

2.16.1.1 Core Definition

- An **operating system (OS)** is a **program** or **piece of software** that comes **pre-installed** on a computer system.
- It is **responsible for managing the hardware** of the computer, including:
 - Processors (CPU)
 - RAM (memory)
 - Storage devices (HDD/SSD)
 - Input/output devices (monitor, keyboard, mouse, printer, etc.)

2.16.1.2 Key Functions

1. Hardware Management

- Computer users **cannot directly manage hardware**.
- The OS acts as an **intermediary** between users and hardware, handling:
 - **Allocation** of hardware resources to running programs.
 - **Control** and **coordination** of hardware operations.

2. Resource Allocation

- Multiple programs run simultaneously on a computer, each requiring hardware resources (CPU time, memory, storage, etc.).
- The OS **dynamically allocates** these resources to programs as needed.

3. User-Hardware Interface

- Users **do not interact directly** with hardware; the OS provides an **abstraction layer** to simplify interactions.
- Example: When a user prints a document, the OS manages the printer hardware, not the user.

4. Persistent Operation

- The OS is the **first program to load** when a computer is powered on (**boot process**).
- It **remains active** as long as the computer is powered on, managing all other programs and hardware.

2.16.2 Types of Operating Systems

2.16.2.1 Classification Based on Computing Environments The type of OS depends on the **computing environment** it is designed for. Four primary categories exist:

2.16.2.1.1 1. Mainframe Operating Systems

- **Purpose:** Designed for **mainframe computers**, which are used for:
 - **Bulk data processing** (e.g., banking transactions, census data).
 - **Heavy computational tasks** (e.g., scientific simulations, large-scale databases).
- **Characteristics:**

- Optimized for **high throughput** and **reliability**.
- Supports **multiple users** simultaneously (multi-user systems).
- Example: IBM z/OS.

2.16.2.1.2 2. Personal Computer (PC) Operating Systems

- **Purpose:** Intended for **everyday use** by individual users on **desktops and laptops**.
- **Characteristics:**
 - User-friendly **graphical interfaces** (GUI).
 - Supports **multitasking** (running multiple applications at once).
 - Example: Microsoft Windows, macOS, Linux distributions.

2.16.2.1.3 3. Handheld Device Operating Systems

- **Purpose:** Designed for **mobile devices** such as:
 - Smartphones.
 - Tablets.
- **Characteristics:**
 - Optimized for **touch interfaces** and **low power consumption**.
 - Supports **mobile-specific features** (e.g., GPS, sensors, app ecosystems).
 - Example: Google Android, Apple iOS.

2.16.2.1.4 4. Embedded Operating Systems

- **Purpose:** Used in **embedded systems**, which are specialized computers integrated into larger devices.
- **Examples of Embedded Devices:**
 - Household appliances (microwave ovens, dishwashers).
 - Automotive systems (engine control units, infotainment systems).
 - Industrial machinery.
- **Characteristics:**
 - **Lightweight** and **real-time** (must respond within strict time constraints).
 - Often **lack a GUI** (headless operation).
 - Example: FreeRTOS, QNX, VxWorks.

2.16.2.2 Key Differences Between OS Types

Type	Target Devices	Primary Use Case	Key Features
Mainframe OS	Mainframe computers	Bulk data processing	High throughput, multi-user, reliability
PC OS	Desktops/Laptops	General-purpose computing	GUI, multitasking, user-friendly
Handheld OS	Smartphones/Tablets	Mobile computing	Touch interface, power efficiency
Embedded OS	Appliances/Automotive	Device-specific control	Real-time, lightweight, headless

2.16.3 Popular Operating Systems in the Current Market

2.16.3.1 Overview The following list highlights **widely used OSES** across different computing environments. Note: This is **not exhaustive** but represents dominant players.

2.16.3.1.1 1. Microsoft Windows

- **Category:** PC Operating System.
- **Usage:** Dominates the **desktop and laptop market**.
- **Features:**
 - **User-friendly GUI** (e.g., Windows 10/11).
 - **Wide software compatibility** (supports most commercial applications).
 - **Regular updates** and security patches.
- **Market Share:** Used by **millions of users worldwide**.

2.16.3.1.2 2. Linux Operating Systems

- **Category:** PC and Server Operating System.
- **Usage:** Popular in **servers, desktops, and development environments**.
- **Characteristics:**
 - **Open-source** (free to use and modify).
 - **Highly customizable** (multiple “flavors” or distributions).
 - **Stable and secure** (commonly used in servers).
- **Popular Distributions:**
 - **Ubuntu:** User-friendly, beginner-oriented.
 - **Fedora:** Cutting-edge features, backed by Red Hat.
 - **Debian:** Stable, community-driven.
 - **openSUSE:** Focuses on sysadmins and developers.
- **Other Variants:** Arch Linux, CentOS, Mint, etc.

2.16.3.1.3 3. Apple macOS

- **Category:** PC Operating System.
- **Usage:** Exclusive to **Apple’s Mac computers** (laptops and desktops).
- **Features:**
 - **Unix-based** (robust and secure).
 - **Seamless integration** with Apple ecosystem (iPhone, iPad, iCloud).
 - **Optimized for creative professionals** (e.g., design, video editing).

2.16.3.1.4 4. Google Android

- **Category:** Handheld Operating System.
- **Usage:** Dominates the **smartphone and tablet market**.
- **Features:**
 - **Open-source** (based on Linux kernel).
 - **Customizable** (manufacturers can modify the OS).
 - **Vast app ecosystem** (Google Play Store).
 - **Supports diverse hardware** (from budget to flagship devices).

2.16.3.1.5 5. Apple iOS

- **Category:** Handheld Operating System.
- **Usage:** Exclusive to **Apple’s iPhones and iPads**.
- **Features:**
 - **Closed-source** (proprietary, controlled by Apple).
 - **Highly optimized** for Apple hardware.
 - **Strong security and privacy** features.

- **Limited customization** compared to Android.

2.16.3.2 Summary Table of Popular OSes

OS Name	Type	Target Devices	Key Strengths
Microsoft Windows	PC OS	Desktops/Laptops	User-friendly, wide software support
Linux	PC/Server OS	Desktops/Servers	Open-source, customizable, secure
macOS	PC OS	Apple Macs	Unix-based, ecosystem integration
Android	Handheld OS	Smartphones/Tablets	Open-source, vast app ecosystem
iOS	Handheld OS	iPhones/iPads	Secure, optimized for Apple hardware

2.16.4 Lecture Summary

1. Definition of an OS:

- A **software program** that manages hardware and acts as an intermediary between users and computer resources.
- **Runs continuously** from boot until shutdown.

2. Types of OS:

- **Mainframe OS:** For bulk data processing (e.g., IBM z/OS).
- **PC OS:** For general-purpose computing (e.g., Windows, macOS, Linux).
- **Handheld OS:** For mobile devices (e.g., Android, iOS).
- **Embedded OS:** For specialized devices (e.g., FreeRTOS in appliances).

3. Popular OS Examples:

- **PC:** Windows, Linux (Ubuntu/Fedora), macOS.
- **Handheld:** Android, iOS.

2.17 Working of a modern computer system

2.17.1 1. Introduction to Modern Computer Systems

- This lecture explores the **components**, **functional aspects**, and **interactions** of a modern computer system.
- The goal is to understand how these components work together to execute processes and perform computations.

2.17.2 2. Key Components of a Modern Computer System

2.17.2.1 2.1 Central Processing Unit (CPU)

- The **CPU (or processor)** is responsible for executing instructions.
- A **process** is defined as a **program in execution**.
- The CPU executes instructions **sequentially**, one by one, within a process.

2.17.2.2 2.2 Main Memory (RAM)

- **Main memory (Random Access Memory, RAM)** stores:
 - **Instructions** to be executed by the CPU.
 - **Data** required for instruction execution.
- The CPU **cannot directly access secondary storage** (e.g., hard drives, SSDs).
 - All instructions and data **must be loaded into main memory** before execution.

2.17.2.3 2.3 Cache Memory

- **Cache memory** is a **small, high-speed memory** located **on the CPU chip**.
- It is **much smaller** than main memory but **significantly faster**.
- **Purpose:**
 - Reduces the **speed mismatch** between the CPU and main memory.
 - Stores **frequently used instructions and data** to minimize access delays.
- **Access time:**
 - Cache memory is accessed **much faster** than main memory.

2.17.2.4 2.4 Input/Output (I/O) Devices

- **I/O devices** handle **input and output operations** for processes.
- **Examples:**
 - Keyboards, mice (input).
 - Monitors, printers (output).
 - Storage devices (both input and output).
- **Role:**
 - Facilitate communication between the computer and external entities.
 - Generate **interrupts** upon completion of I/O operations.

2.17.3 3. Interrupts: Mechanism and Functionality

2.17.3.1 3.1 Definition of an Interrupt

- An **interrupt** is a **signal sent to the CPU** to notify it of an **event**.
- **Purpose:**
 - Inform the CPU about the **completion or occurrence of an event** (e.g., I/O operation completion).

2.17.3.2 3.2 Types of Interrupts

1. **Hardware Interrupts**
 - Generated by **I/O devices** (e.g., keyboard press, disk read completion).
2. **Software Interrupts**
 - Generated by **software programs** (e.g., system calls, exceptions).

2.17.3.3 3.3 Example: I/O Operation Completion

- **Scenario:**
 - A process issues an **I/O request** (e.g., reading data from a disk).
 - The I/O device processes the request.
 - Upon completion, the device sends an **interrupt** to the CPU.
- **CPU Response:**
 - Pauses current execution.
 - Handles the interrupt (e.g., resumes the process that issued the I/O request).

2.17.3.4 3.4 Problem with Frequent Interrupts

- **Issue:**
 - In **bulk data transfers** (e.g., transferring 100 GB of data), if an interrupt is generated **per byte**, the CPU is **constantly interrupted**, leading to:
 - * **High overhead.**

* **Severe performance degradation.**

- **Solution: Direct Memory Access (DMA)** (discussed in Section 4).

2.17.4 4. Direct Memory Access (DMA)

2.17.4.1 4.1 Definition and Purpose

- **DMA** is a mechanism that allows **I/O devices to transfer data directly to/from main memory without continuous CPU involvement.**
- **Goal:**
 - Reduce CPU overhead by **bypassing the CPU** during data transfer.
 - Generate **only one interrupt** after the **entire transfer is complete.**

2.17.4.2 4.2 How DMA Works

1. **Initialization Phase:**
 - The CPU **sets up the DMA controller** with:
 - Source and destination addresses.
 - Transfer size.
2. **Data Transfer Phase:**
 - The **DMA controller** manages the transfer **independently.**
 - The CPU is **not interrupted** during the transfer.
3. **Completion Phase:**
 - After the **entire transfer is complete**, the DMA controller sends a **single interrupt** to the CPU.

2.17.4.3 4.3 Advantages of DMA

- **Reduces CPU load** by eliminating per-byte interrupts.
- **Improves system performance** for large data transfers.
- **Efficient for bulk operations** (e.g., disk I/O, network transfers).

2.17.5 5. Interaction of Computer Components

2.17.5.1 5.1 CPU and Memory Interaction

1. **Instruction Fetch:**
 - The CPU fetches **instructions** from **main memory** (or cache).
2. **Data Access:**
 - If an instruction requires data, the CPU fetches it from **main memory** (or cache).
3. **Execution:**
 - The CPU executes the instruction, possibly **modifying data.**
4. **Result Storage:**
 - Results are stored back into **main memory.**

2.17.5.2 5.2 CPU and I/O Devices Interaction

1. **I/O Request Generation:**
 - A process executing on the CPU issues an **I/O request** (e.g., read from disk).
2. **Data Transfer:**
 - **Without DMA:**
 - Data is transferred **byte by byte**, with an **interrupt per byte.**
 - **With DMA:**

- The **DMA controller** handles the transfer **without CPU intervention**.
- Only **one interrupt** is generated upon completion.

3. Interrupt Handling:

- The CPU **pauses current execution** to handle the interrupt.
- Resumes the process after the I/O operation completes.

2.17.5.3 5.3 Role of Cache Memory

- **Frequently accessed instructions/data** are stored in the **cache**.
- **Reduces main memory access time**, improving performance.
- The CPU **first checks the cache** before accessing main memory.

2.17.6 6. Holistic View of Computer System Operation

1. Process Execution:

- The CPU executes instructions **sequentially** from main memory (or cache).

2. Memory Hierarchy:

- **Cache** (fastest, smallest) -> **Main Memory** (slower, larger) -> **Secondary Storage** (slowest, largest).

3. I/O Operations:

- Processes generate I/O requests handled by **I/O devices**.
- **Interrupts** notify the CPU of I/O completion.
- **DMA** optimizes bulk data transfers by minimizing CPU involvement.

4. Performance Optimization:

- **Cache memory** reduces main memory access latency.
- **DMA** reduces CPU overhead in I/O operations.

2.17.7 7. Summary of Key Concepts

Component	Function	Key Feature
CPU	Executes instructions sequentially.	Core processing unit.
Main Memory (RAM)	Stores instructions and data for CPU access.	Volatile, fast access.
Cache Memory	Small, ultra-fast memory on the CPU chip for frequent data/instructions.	Reduces main memory access time.
I/O Devices	Handle input/output operations (e.g., disk, keyboard, monitor).	Generate interrupts on completion.
Interrupts	Signals to CPU indicating event completion (e.g., I/O done).	Can be hardware or software-generated.
DMA	Enables direct memory-I/O transfer without per-byte CPU interrupts.	Improves bulk data transfer efficiency.

2.17.8 8. Conclusion

- A modern computer system consists of **interdependent components** (CPU, memory, I/O devices, cache) that work together to execute processes.
- **Interrupts** and **DMA** play crucial roles in **efficient I/O handling**.
- **Cache memory** bridges the **speed gap** between CPU and main memory.
- Understanding these interactions provides a **holistic view** of how computers function at a low level.